

5. LVGL Development

5. LVGL Development

5.1 Program Download Instructions (Must Read)

5.2 Introduction to LVGL

5.2.1 What is LVGL?

5.2.2 LVGL Resource Requirements

5.2.3 LVGL Architecture Composition

5.3 PC Environment Setup Experiment

5.3.1 LVGL Simulator Project Download

5.3.2 Visual Studio Environment Setup

5.3.3 Opening the Visual Studio Simulator Project

5.3.4 Opening the Windows Simulator Project

5.4 LVGL Mouse Input Support Experiment

5.4.1 Experiment Introduction

5.4.2 Program Download

5.4.3 Program Outcome

5.4.4 Program Analysis

5.5 LVGL File System Experiment

5.5.1 Experiment Introduction

5.5.2 Program Download

5.5.3 Program Outcome

5.5.4 Program Analysis

5.6 LVGL Built-in Font Application Experiment

5.6.1 Experiment Introduction

5.6.2 Program Download

5.6.3 Program Outcome

5.6.4 Program Analysis

5.7 LVGL External Font Loading and Display Experiment

5.7.1 Experiment Introduction

5.7.2 Program Download

5.7.3 Program Outcome

5.7.4 Program Analysis

5.8 LVGL TrueType Font Support Experiment

5.8.1 Experiment Introduction

5.8.2 Program Download

5.8.3 Program Outcome

5.8.4 Program Analysis

5.9 LVGL Basic Objects and Widgets Experiment

5.9.1 Experiment Introduction

5.9.2 Program Download

5.9.3 Program Outcome

5.9.4 Program Analysis

5.10 LVGL Arc Widget Application Experiment

5.10.1 Experiment Introduction

5.10.2 Program Download

5.10.3 Program Outcome

5.10.4 Program Analysis

5.11 LVGL Embedded Bootloader Design Experiment

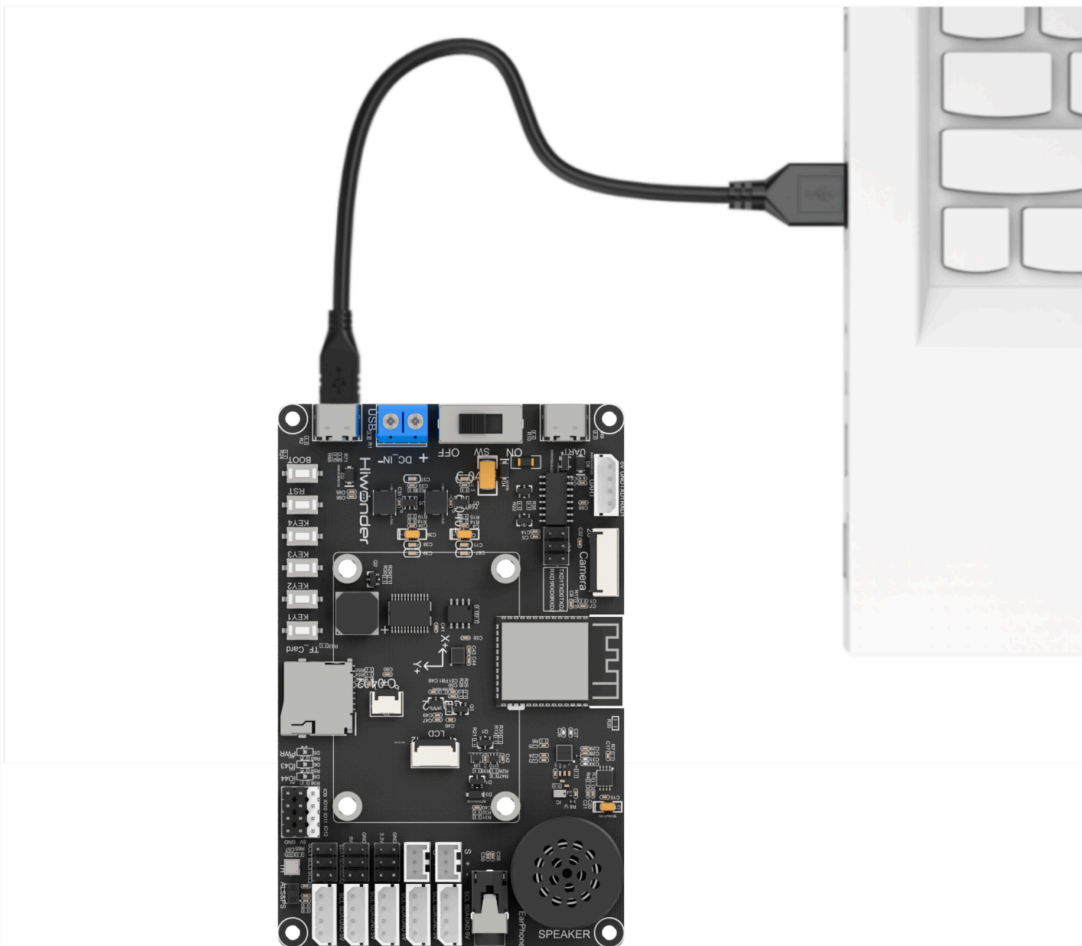
5.11.1 Experiment Introduction

5.11.2 Program Download

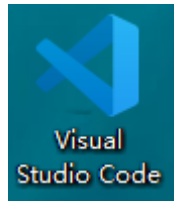
- 5.11.3 Program Outcome
- 5.11.4 Program Analysis
- 5.12 LVGL Custom Drawing Experiment
 - 5.12.1 Experiment Introduction
 - 5.12.2 Program Download
 - 5.12.3 Program Outcome
 - 5.12.4 Program Analysis
- 5.13 LVGL File System Depth Application Experiment
 - 5.13.1 Experiment Introduction
 - 5.13.2 Program Download
 - 5.13.3 Program Outcome
 - 5.13.4 Program Analysis
- 5.14 LVGL Data Format Display Conversion Experiment
 - 5.14.1 Experiment Introduction
 - 5.14.2 Program Download
 - 5.14.3 Program Outcome
 - 5.14.4 Program Analysis

5.1 Program Download Instructions (Must Read)

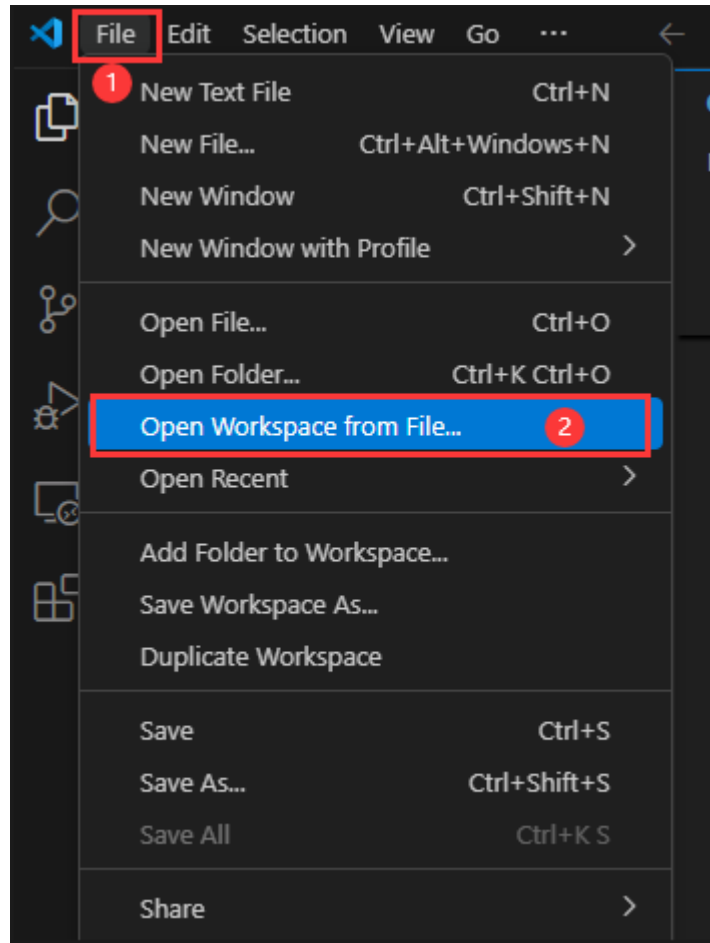
1. Connect the UART interface of the ESP32-S3 development board to the computer using a Type-C cable.



2. Double-click to open the **Visual Studio Code** software.



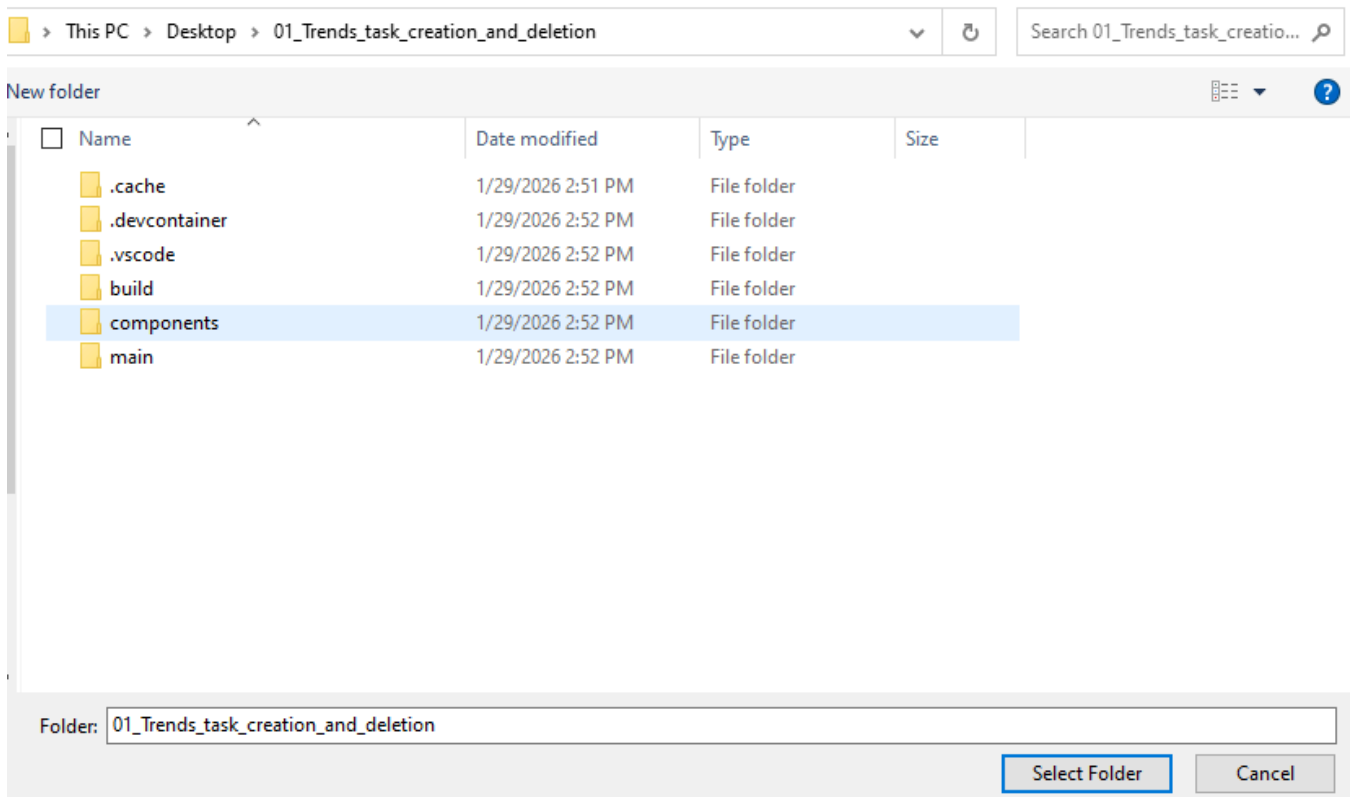
3. Click **File** -> **Open Workspace from File...** in the top left corner of Visual Studio Code.



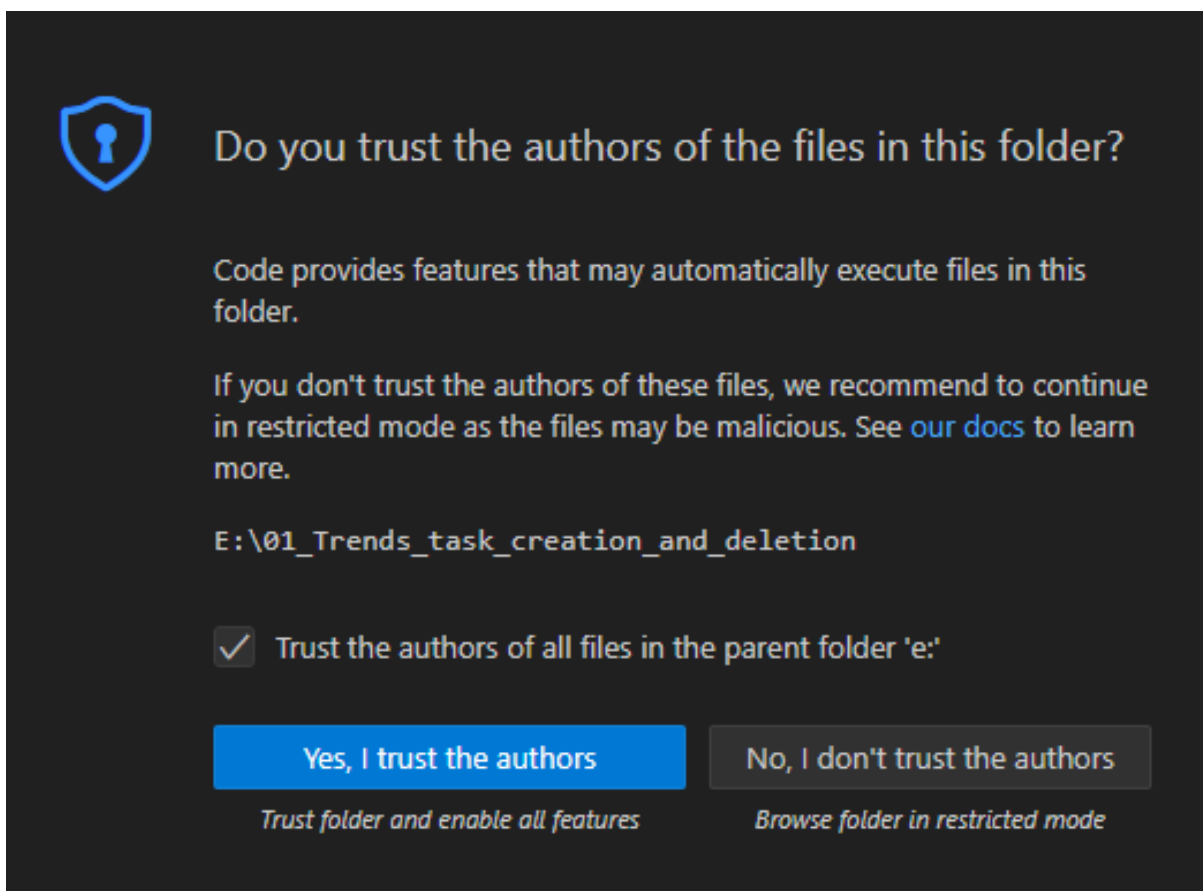
4. Taking [Source Code](#) -> **01_Trends_task_creation_and_deletion** under the same path as this document as an example, click **Select Folder**.

Note

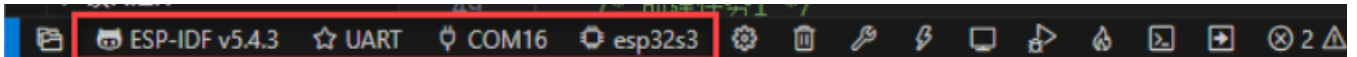
Ensure the program file path contains only English characters, as non-English characters may cause compilation failures.



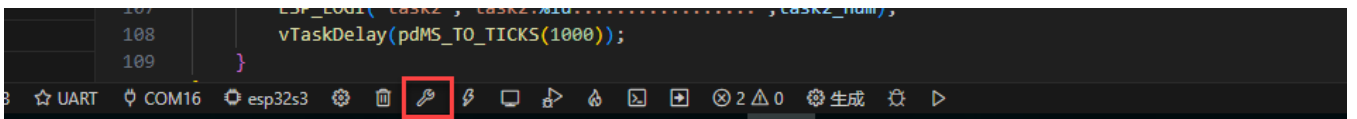
After opening the folder, if a Visual Studio Code pop-up appears, check **Trust the authors of all files in the parent folder 'Source Code'**, and click **Yes, I trust the authors** to open the program files.



5. In the bottom left corner of the page, select the ESP-IDF version **ESP-IDF v5.4.3**, the flash method **UART**, the port number **COM16**, and the chip model **esp32s3**. Use the actual port number, as COM16 is used here as an example.

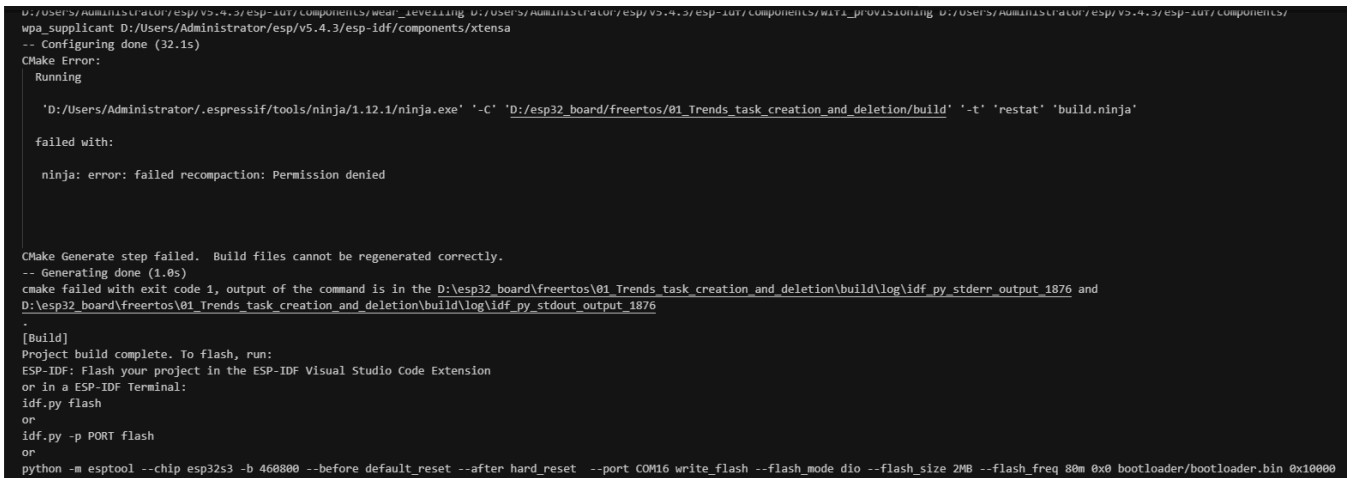


6. Click the icon inside the red frame in the figure below to compile the project.

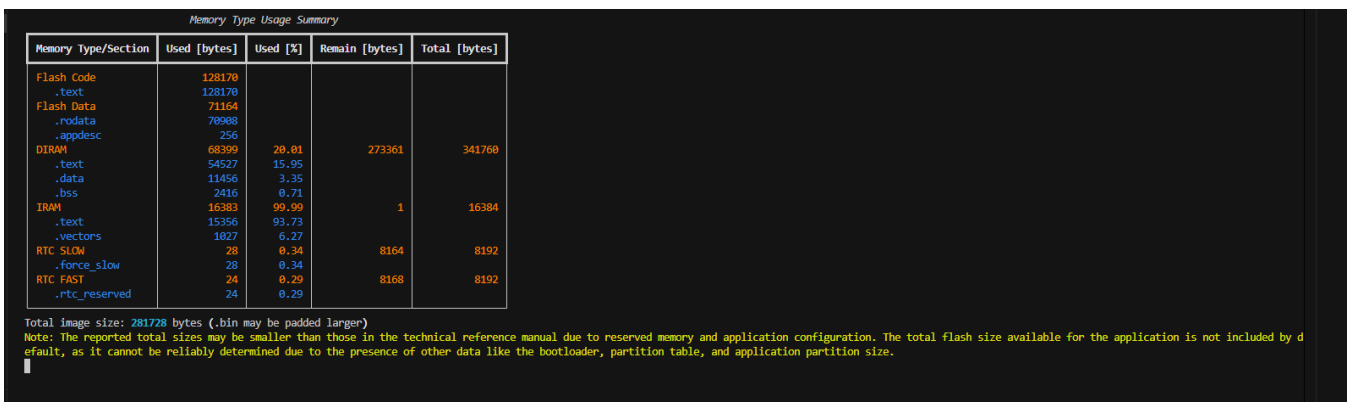


Note

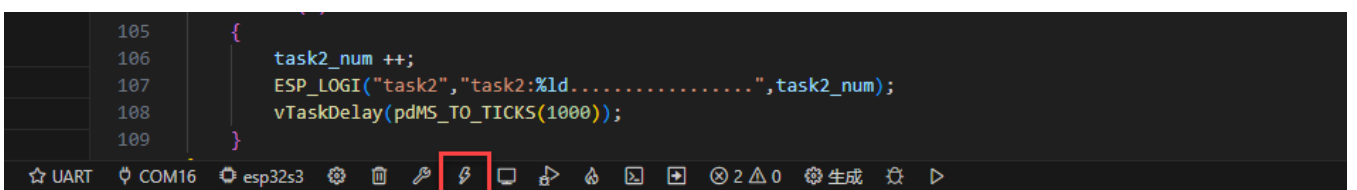
Compilation takes some time, please wait for it to complete.



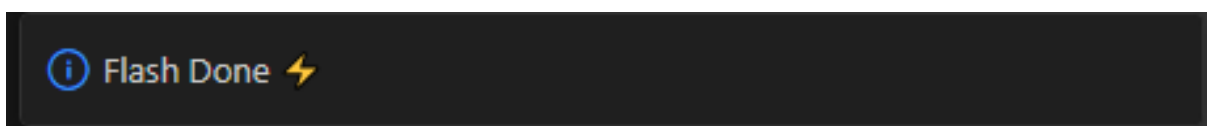
When the terminal displays the page shown below, compilation is complete.



7. After compilation is complete, click the icon inside the red frame in the figure below to download the program to the development board.



When the pop-up shown below appears, the program has been successfully flashed.



5.2 Introduction to LVGL

5.2.1 What is LVGL?

LVGL, fully known as Light and Versatile Graphics Library, is a lightweight, embeddable GUI library specifically designed for embedded systems. It is an open-source project providing rich graphical interface elements and functions, suitable for creating various user interfaces ranging from small embedded devices to large instrument control panels. It possesses the following characteristics:

- **Rich modular graphical components:** Buttons, charts, lists, sliders, images, etc.
- **Advanced graphics engine:** Animations, anti-aliasing, opacity, smooth scrolling, layer blending effects, etc.
- **Support for multiple input devices:** Touch screens, keyboards, encoders, buttons, etc.
- **Support for multiple display devices.**
- **Hardware independence:** Can run on any display.
- **Configurable and scalable:** Minimum resource usage: 64 kB Flash, 16 kB RAM.
- **CSS-like design:** Supports designing and laying out graphical interfaces in a CSS-like manner.
- **System support:** Supports operating systems, external memory, and hardware acceleration.
- **Written in C.**

5.2.2 LVGL Resource Requirements

To run LVGL smoothly, the minimum requirements are as follows:

- 16, 32, or 64-bit microcontroller or processor.
- Clock speed greater than 16MHz is recommended.
- **Flash/ROM:** Minimum 64KB, greater than 180KB recommended.
- **RAM:**
 - Static RAM usage:** Approximately 8-16KB depending on used features and object types.
 - Stack:** Minimum greater than 2KB, greater than 4KB recommended.
 - Dynamic data (heap):** Minimum greater than 4KB. If using several objects, a minimum greater than 16KB.
 - Display buffer:** At least greater than "horizontal resolution pixels", greater than 10 times "horizontal resolution pixels" recommended.
- C99 or newer compiler.
- Basic C language knowledge: Pointers, callbacks, structures, etc.

5.2.3 LVGL Architecture Composition

LVGL primarily consists of the following modules:

- **Display Driver Layer:** Responsible for refreshing the screen and driving display devices like LCD/OLED.
- **Input Driver Layer:** Responsible for handling touch and key input events.
- **Core Module:** Widget tree, layout, event system.

- **Widgets Library:** A collection of all UI elements.
- **Draw Module:** Renders pixels, lines, images, fonts, etc.

5.3 PC Environment Setup Experiment

5.3.1 LVGL Simulator Project Download

A Visual Studio simulator project and a Windows simulator project are provided in the [Source Code\03_PC_lvgl](#) path under the same directory as this document. Simply unzip to use.

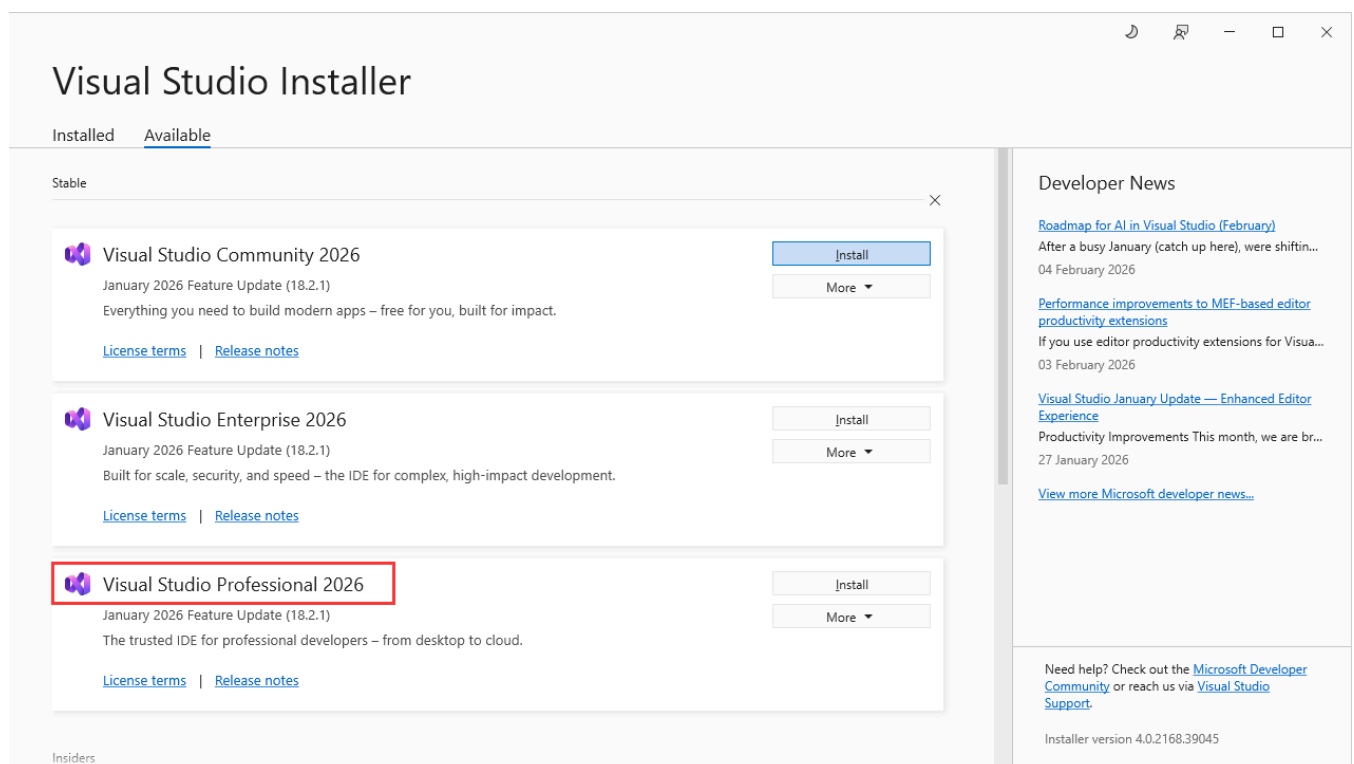


5.3.2 Visual Studio Environment Setup

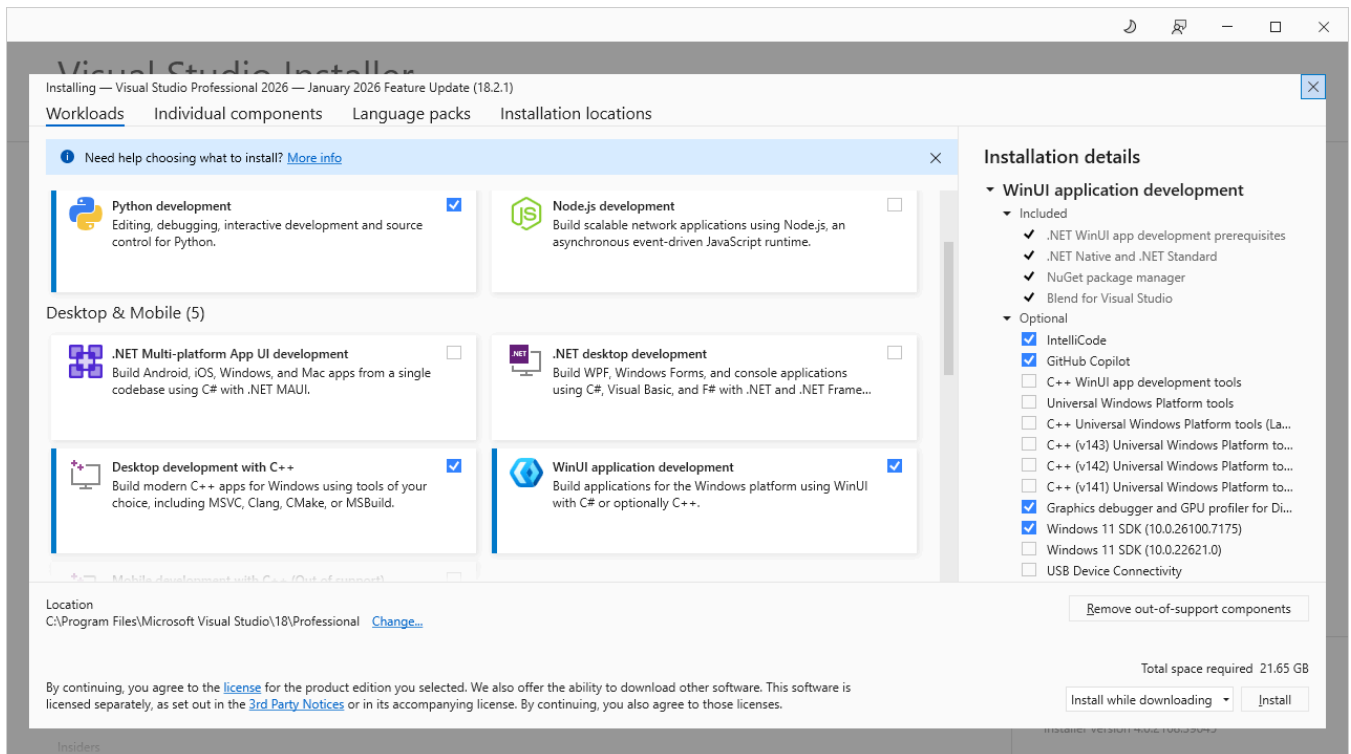
Note

If Visual Studio is already installed and the environment is configured, this section can be ignored. Skip to section [5.3.3 Opening the Visual Studio Simulator Project](#).

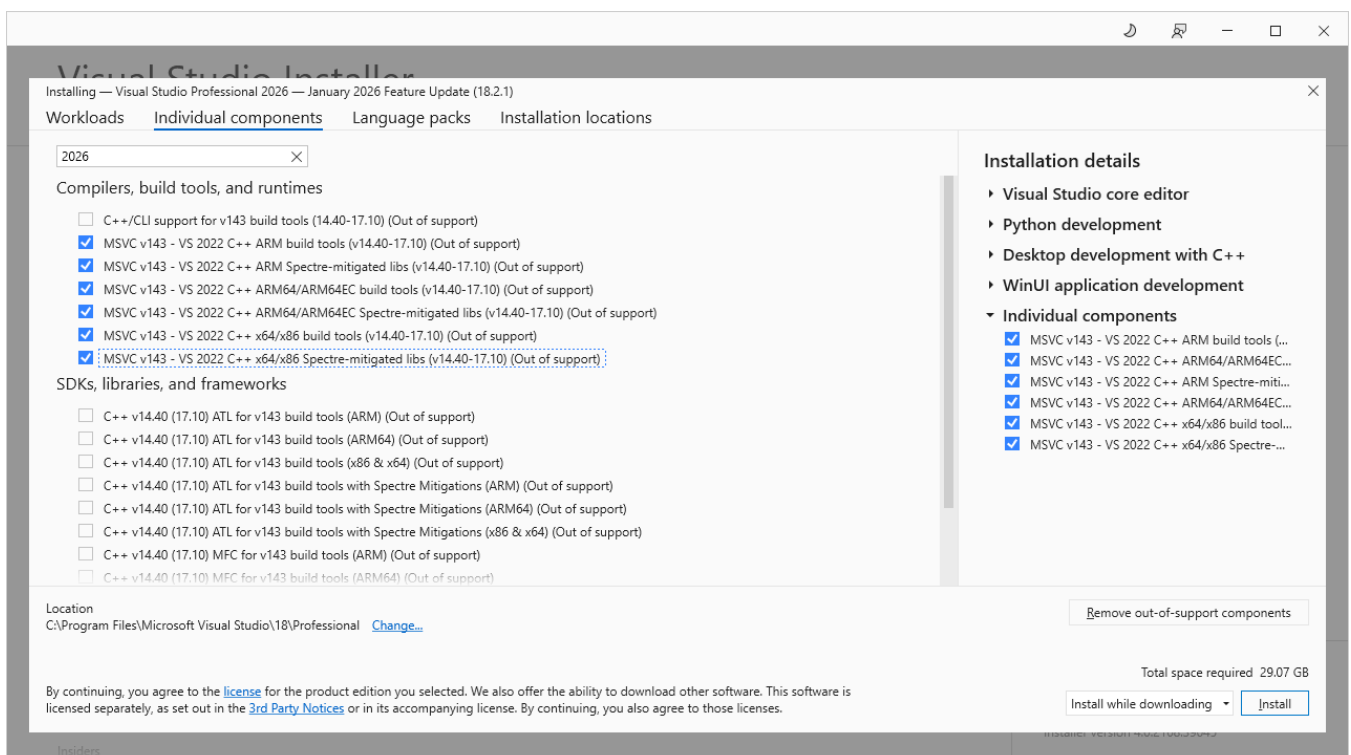
1. Click the link and wait briefly. The browser will automatically start downloading the Visual Studio 2026 installation tool named **VisualStudioSetup.exe**. [Click to download Visual Studio 2026 Installer](#).
2. Open the downloaded installation tool, enter the **Visual Studio Installer** interface, find the **Visual Studio Community 2026** option, and click **Install**, as shown below:



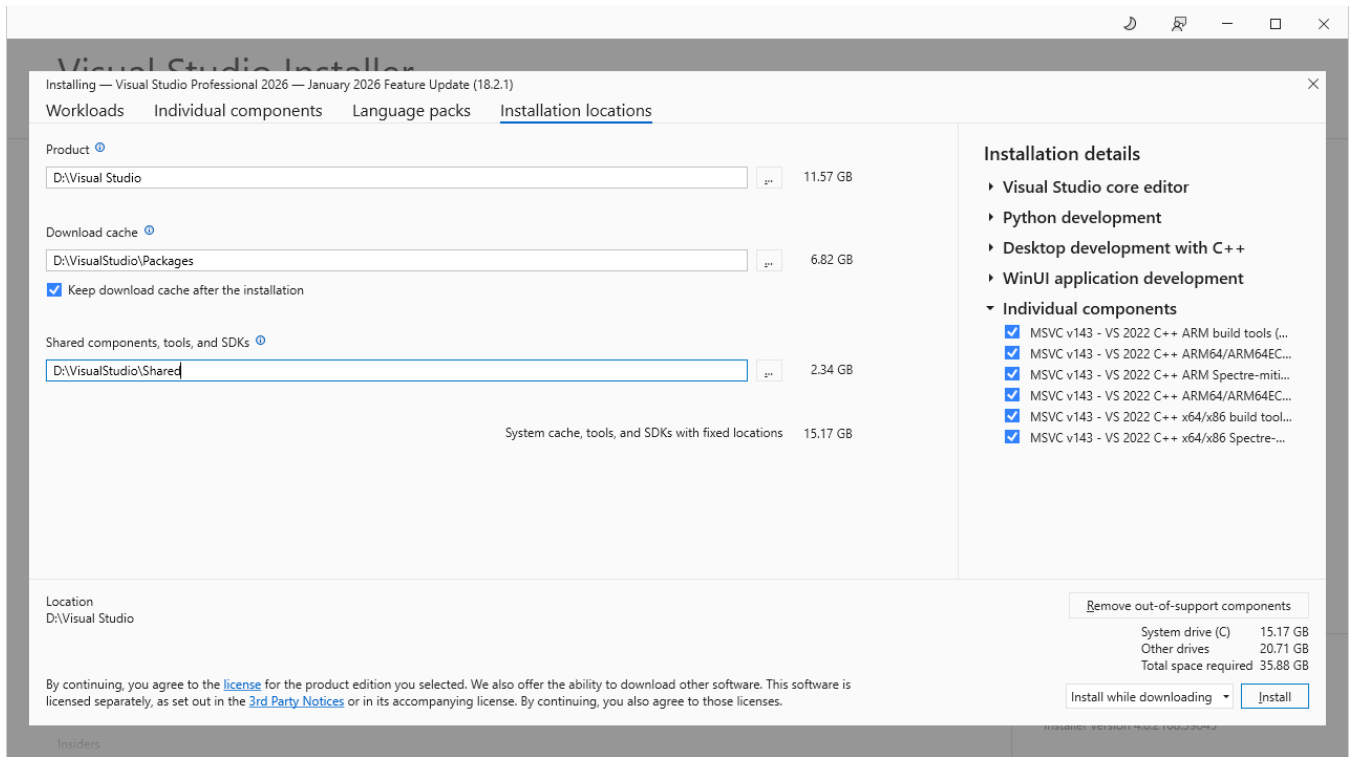
3. In the Workloads interface, check **Python development**, **Desktop development with C++**, and **WinUI application development**. After checking, click **Individual components** at the top:



4. Enter **2026** in the search bar, scroll to the bottom to see **Compilers, build tools, and runtimes**, and check the options as shown in the figure below:



5. Click **Installation locations** at the top. Since Visual Studio 2026 occupies a large amount of memory, it is not recommended to install it on the C drive. Modify the corresponding path by referring to the figure below.

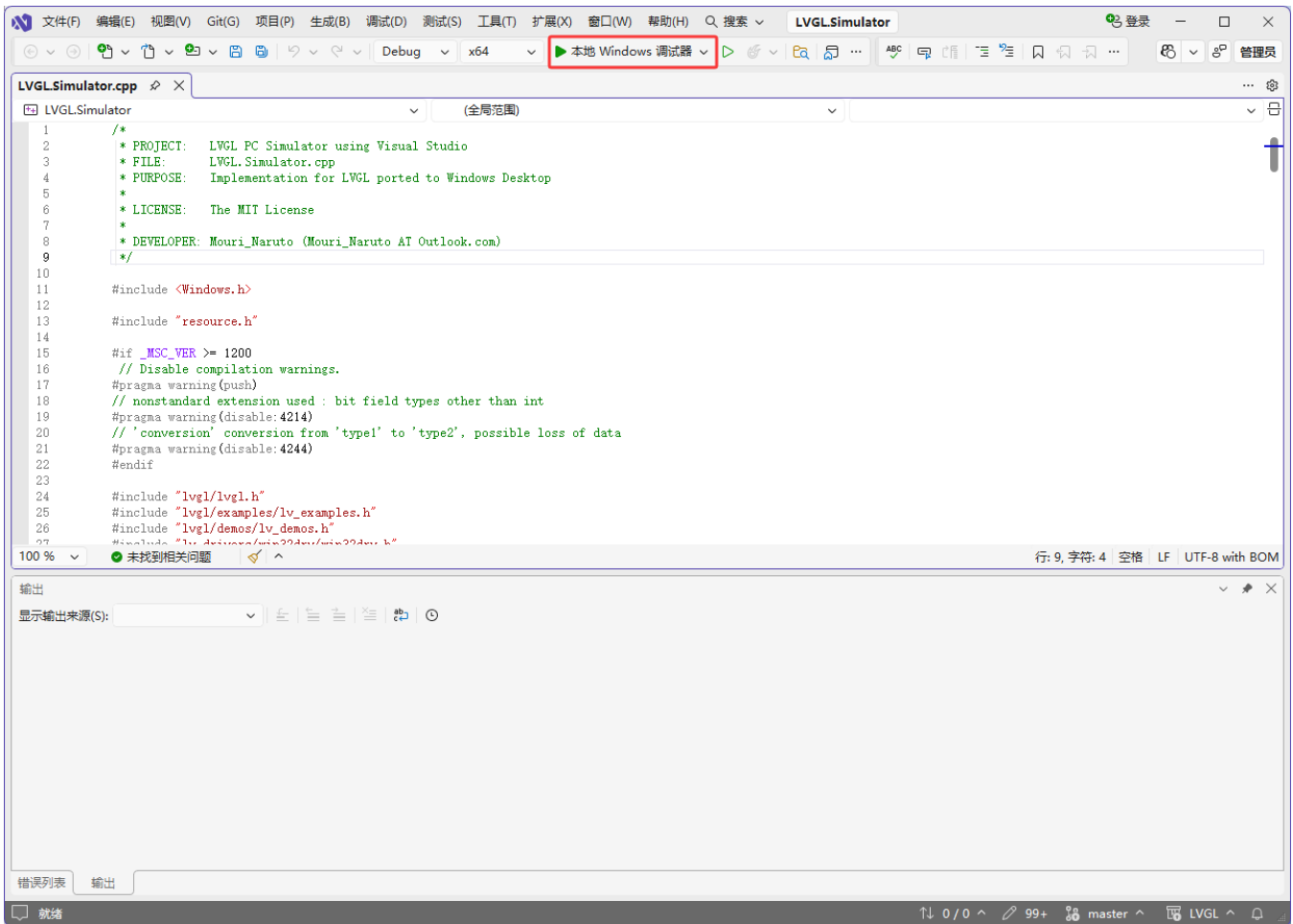


5.3.3 Opening the Visual Studio Simulator Project

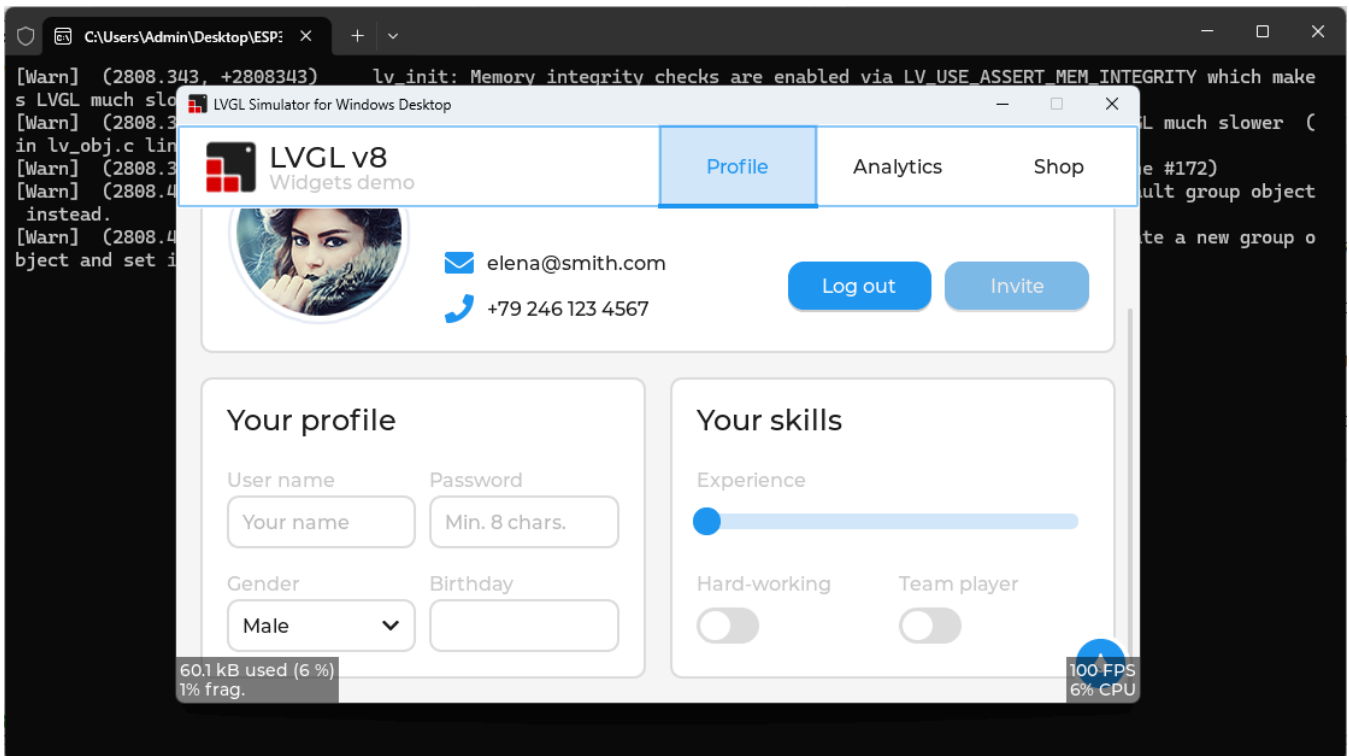
1. Open the **LVGL.Simulator.sln** file in the root directory of the Visual Studio simulator project folder **lv_port_win_visual_studio-master**.

📁 .github	2025/12/16 16:30	文件夹	
📁 .vs	2025/12/16 16:30	文件夹	
📁 build	2025/12/15 14:52	文件夹	
📁 LVGL.Simulator	2025/12/16 16:31	文件夹	
📁 Output	2025/12/16 16:31	文件夹	
⚙️ .editorconfig	2022/7/6 22:31	Editor Config 源...	1 KB
⚙️ .gitignore	2022/7/6 22:31	Git Ignore 源文件	7 KB
📄 .gitmodules	2022/7/6 22:31	GITMODULES 文...	1 KB
📄 BuildAllTargets.cmd	2022/7/6 22:31	Windows 命令脚本	1 KB
📄 BuildAllTargets.proj	2022/7/6 22:31	PROJ 文件	3 KB
📄 Directory.Build.props	2022/7/6 22:31	Project Property...	3 KB
📄 LICENSE	2022/7/6 22:31	文件	2 KB
📄 LVGL.Simulator.sln	2025/12/15 14:28	Microsoft Visual...	2 KB
📄 README.md	2022/7/6 22:31	Markdown File	6 KB
📄 Screenshot.png	2022/7/6 22:31	PNG 文件	87 KB

2. After entering the Visual Studio interface, click **Local Windows Debugger**:

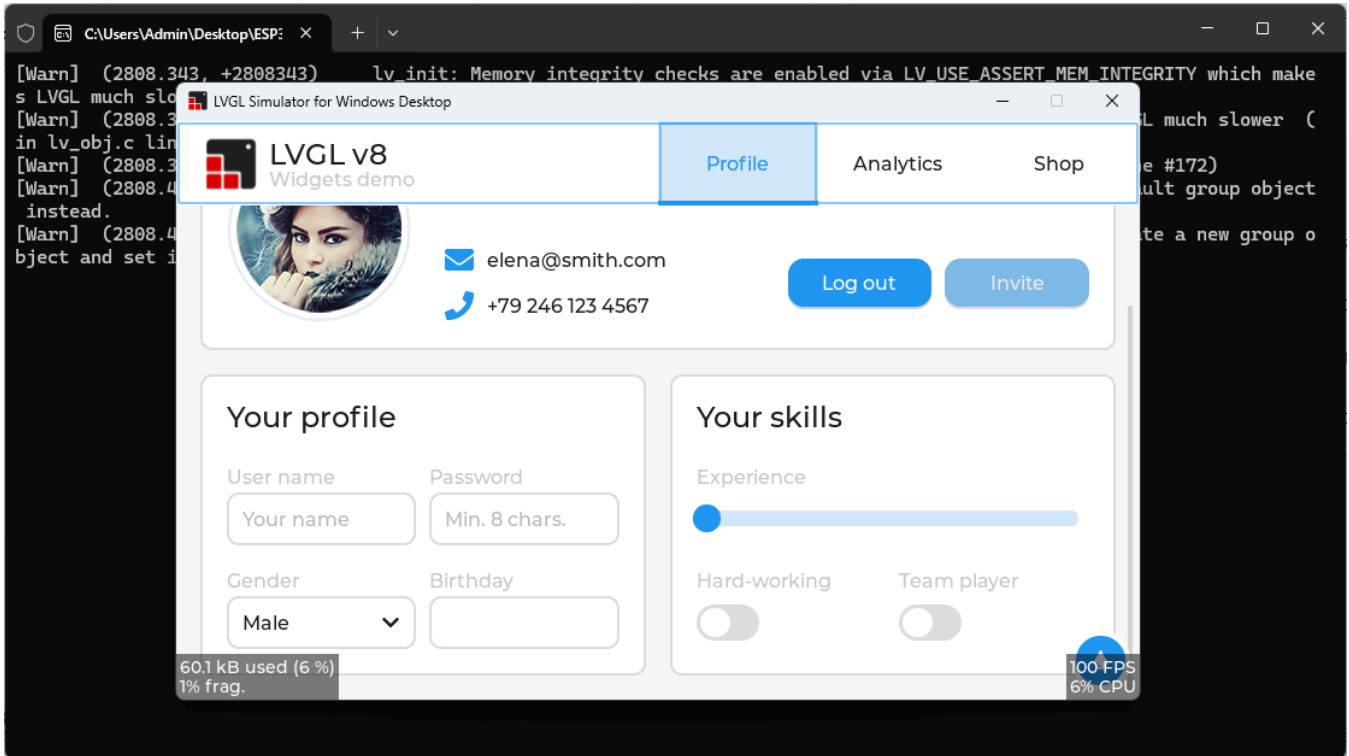


3. The effect after successful execution is as follows:



5.3.4 Opening the Windows Simulator Project

1. Open the **LittleGL.exe** file in the **bin\Debug** path of the Windows simulator project folder **lv_port_win_codeblocks-master** to execute. The execution effect is as follows:



5.4 LVGL Mouse Input Support Experiment

5.4.1 Experiment Introduction

This experiment explores how to simulate a mouse using a touch screen to verify and understand the working mechanism of LVGL pointer-type input devices.

5.4.2 Program Download

Locate [Source Code](#) -> **04_lvgl_add_mouse** under the same path as this document and click **Select Folder**.

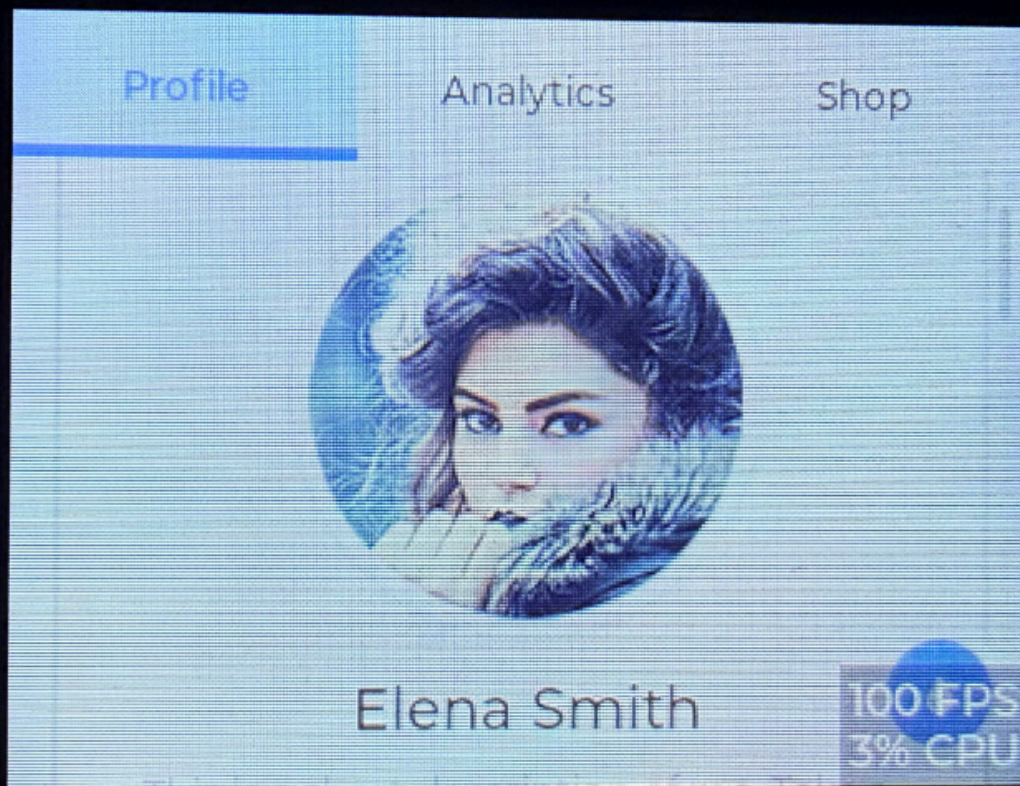
Note

Ensure the program file path contains only English characters, as non-English characters may cause compilation failures.

For specific program download tutorials, refer to [5.1 Program Download Instructions \(Must Read\)](#).

5.4.3 Program Outcome

After downloading the program and powering on the development board, a pattern similar to a mouse cursor will appear when touching the screen.



5.4.4 Program Analysis

- **lvgl_demo.c** file

1. `lvgl_demo()` is the main entry function for the LVGL demonstration program. It first initializes the LVGL core library, display interface, and input interface. Then, it uses the ESP32 timer to create and start a time-base timer with a 1ms period to provide timestamps for LVGL. Next, it enables the official LVGL widget demo. Finally, it periodically calls the LVGL core processing function within a loop, combined with FreeRTOS delays, to ensure the continuous operation of LVGL event handling and interface refreshing.

```
void lvgl_demo(void)
{
    lv_init();                /* Initialize LVGL graphics library */
    lv_port_disp_init();      /* Initialize LVGL display interface, placed after lv_init() */
    lv_port_indev_init();     /* Initialize LVGL input interface, placed after lv_init() */

    /* Provide time base unit for LVGL */
    const esp_timer_create_args_t lvgl_tick_timer_args = {
        .callback = &increase_lvgl_tick,
        .name = "lvgl_tick"
    };
    esp_timer_handle_t lvgl_tick_timer = NULL;
    ESP_ERROR_CHECK(esp_timer_create(&lvgl_tick_timer_args, &lvgl_tick_timer));
    ESP_ERROR_CHECK(esp_timer_start_periodic(lvgl_tick_timer, 1 * 1000));
}
```

```

/* Official demo, requires enabling the corresponding Demo in SDK Configuration */
// /* lv_demo_stress */();
lv_demo_widgets();

while (1)
{
    lv_timer_handler();          /* LVGL Timer Handler */
    vTaskDelay(pdMS_TO_TICKS(10)); /* Delay 10 milliseconds */
}
}

```

2. `lv_port_disp_init()` is the adaptation and registration function for the LVGL display device. It initializes the physical LCD hardware, allocates a double display buffer using DMA-compatible memory, initializes the LVGL display buffer object and driver descriptor, configures key parameters such as screen resolution, flush callback function, and LCD panel handle, and finally registers the adapted display driver to LVGL, enabling the connection between LVGL and the physical LCD screen.

```

void lv_port_disp_init(void)
{
    void *buf1 = NULL;
    void *buf2 = NULL;

    lcd_init();

    // Declare as static
    static lv_disp_draw_buf_t disp_buf; // Holds display buffer information
    static lv_disp_drv_t disp_drv;     // Display driver descriptor

    buf1 = heap_caps_malloc(lcddev.width * 60 * sizeof(lv_color_t), MALLOC_CAP_DMA);
    buf2 = heap_caps_malloc(lcddev.width * 60 * sizeof(lv_color_t), MALLOC_CAP_DMA);

    lv_disp_draw_buf_init(&disp_buf, buf1, buf2, lcddev.width * 60);

    lv_disp_drv_init(&disp_drv);
    disp_drv.hor_res = lcddev.width;
    disp_drv.ver_res = lcddev.height;
    disp_drv.flush_cb = lvgl_disp_flush_cb; // Use the static function declared earlier
    disp_drv.draw_buf = &disp_buf;
    disp_drv.user_data = lcddev.lcd_panel_handle;

    lv_disp_drv_register(&disp_drv);
}

```

3. `lv_port_indev_init()` is the initialization and registration function for the LVGL input device. It comments out the adaptation logic for a standard touchscreen input device and instead initializes a mouse device simulated via the touchscreen. It first initializes the mouse hardware, then configures the LVGL pointer-type input driver type and data read callback function. After registering the mouse driver, it creates a cursor with a HOME symbol icon and binds it to the mouse input device.

```

void lv_port_indev_init(void)
{
    /* Initialize mouse (if any) */
    mouse_init();

    /* Register mouse input device */
    static lv_indev_drv_t indev_drv;
    lv_indev_drv_init(&indev_drv);

    /* Configure input device type */
    indev_drv.type = LV_INDEV_TYPE_POINTER;

    /* Set input device read callback function */
    indev_drv.read_cb = mouse_read;

    /* Register driver in LVGL and save the created input device object */
    lv_indev_t * indev_mouse;
    indev_mouse = lv_indev_drv_register(&indev_drv);

    /* Set cursor. For simplicity, set as a HOME symbol */
    lv_obj_t * mouse_cursor = lv_img_create(lv_scr_act());
    lv_img_set_src(mouse_cursor, LV_SYMBOL_HOME);
    lv_indev_set_cursor(indev_mouse, mouse_cursor);
}

```

4. `lvgl_disp_flush_cb` is the core callback function for LVGL display flushing, responsible for flushing pixel data from the internal LVGL buffer to the physical LCD screen. It retrieves the LCD panel handle from the display driver, calls the ESP32 LCD driver's bitmap drawing interface to draw the color data of the specified area to the screen, and finally notifies LVGL that the flush is complete. This is the critical step for rendering LVGL graphics data onto the hardware.

```

static void lvgl_disp_flush_cb(lv_disp_drv_t *drv, const lv_area_t *area, lv_color_t
*color_map)
{
    esp_lcd_panel_handle_t panel_handle = (esp_lcd_panel_handle_t)drv->user_data;
    esp_lcd_panel_draw_bitmap(panel_handle, area->x1, area->y1, area->x2 + 1, area->y2 + 1,
color_map);
    lv_disp_flush_ready(drv);
}

```

5. `increase_lvgl_tick` is the callback function for the ESP32 periodic timer. Its function is to provide a time base for LVGL. Each time it is called, `lv_tick_inc(1)` informs LVGL that 1 millisecond has elapsed, supporting LVGL's animations, timers, delays, and other time-related functions.

```

static void increase_lvgl_tick(void *arg)
{
    /* Tell LVGL how many milliseconds have passed */
    lv_tick_inc(1);
}

```

6. `mouse_init` is the initialization function for the mouse device. It actually completes the hardware initialization of the mouse simulated based on the touchscreen by calling the touchscreen device initialization interface.

```
void mouse_init(void)
{
    tp_dev.init();
}
```

1. `mouse_is_pressed` is a function to detect the mouse press state. It scans the touchscreen device state to determine if the touchscreen is pressed, thereby simulating a mouse press event, and returns the corresponding boolean state.

```
static bool mouse_is_pressed(void)
{
    tp_dev.scan();

    if (tp_dev.sta & TP_PRES_DOWN)
    {
        return true;
    }

    return false;
}
```

7. `mouse_get_xy` is the function for reading mouse coordinates. When the mouse (touchscreen) is pressed, it reads the raw coordinates of the touchscreen, swaps and inverts them to convert them into mouse coordinates adapted for the screen display, and returns them to the caller via pointer parameters.

```
static void mouse_get_xy(lv_coord_t *x, lv_coord_t *y)
{
    // Swap x and y coordinates
    (*x) = tp_dev.y[0]; // Change x to y
    (*y) = lcddev.height - tp_dev.x[0]; // Change y to x
}
```

8. `mouse_read` is the core callback function for LVGL mouse input. It is responsible for providing the current mouse state and coordinate data to LVGL. It first calls the coordinate reading function to obtain the mouse position, then detects the mouse press state and sets the corresponding input device state, finally passing this data to LVGL.

```
void mouse_read(lv_indev_drv_t *indev_drv, lv_indev_data_t *data)
{
    // Get current x, y coordinates
    mouse_get_xy(&data->point.x, &data->point.y);

    // Check if mouse button is pressed or released
    if(mouse_is_pressed())
    {
        data->state = LV_INDEV_STATE_PR;
    }
}
```

```
}  
else  
{  
    data->state = LV_INDEV_STATE_REL;  
}  
}
```

5.5 LVGL File System Experiment

5.5.1 Experiment Introduction

This experiment combines LVGL with the FreeRTOS real-time operating system to implement graphical interface rendering, touchscreen interaction, SD card hardware detection, and system status indication verification. The experiment focuses on completing the transplantation of the LVGL graphics library and the adaptation of low-level interfaces. The FreeRTOS multi-task scheduling mechanism is used to separate the LVGL business logic from the LED status indication task, while SD card hardware detection is implemented based on the SPI interface.

5.5.2 Program Download

Locate [Source Code](#)->05_lvgl_fs_use under the same path as this document and click **Select Folder**.

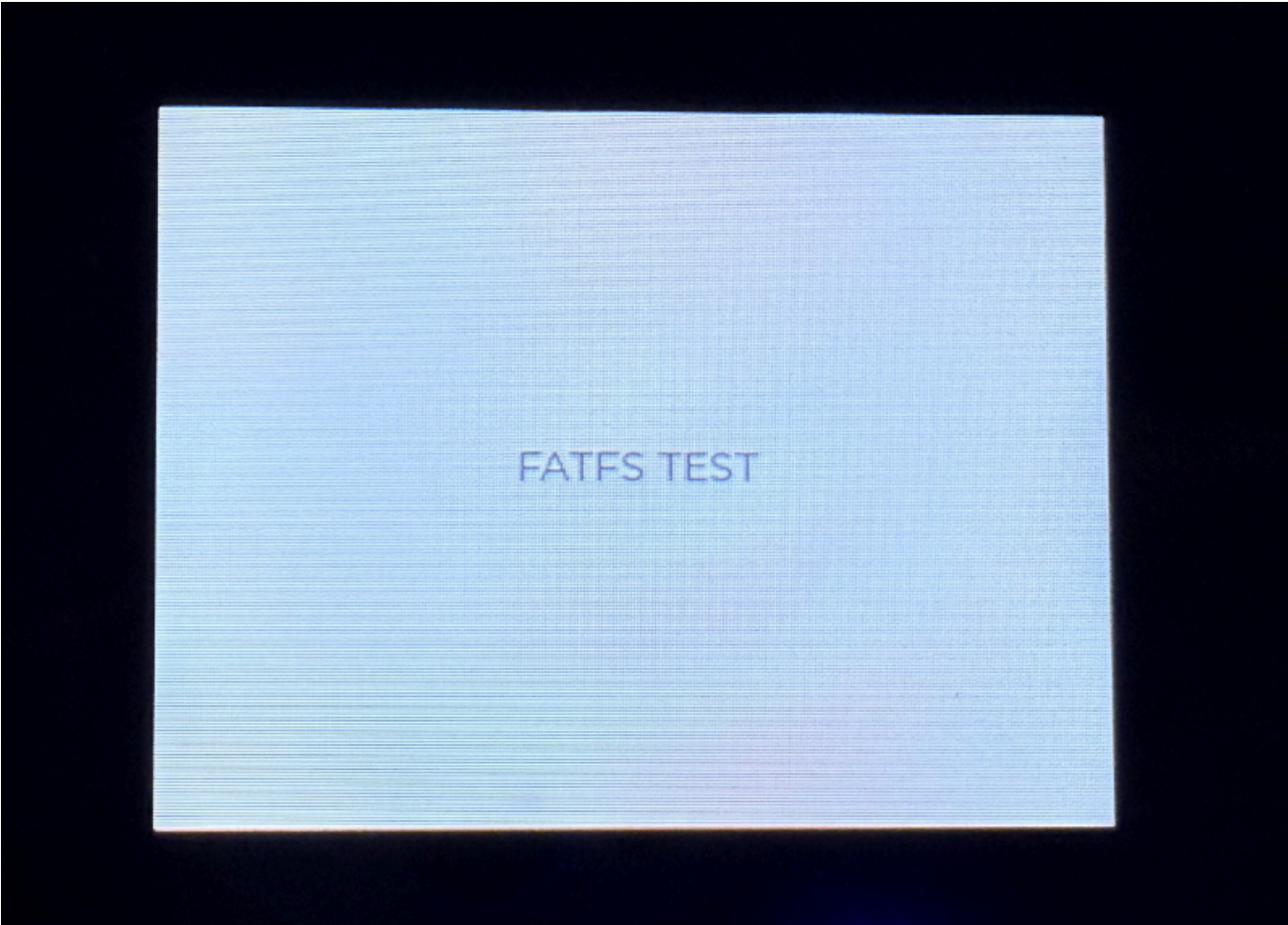
Note

Ensure the program file path contains only English characters, as non-English characters may cause compilation failures.

For specific program download tutorials, refer to [5.1 Program Download Instructions \(Must Read\)](#).

5.5.3 Program Outcome

After the system is powered on, the SD card is detected first. If detection fails, the LCD loops a red error prompt. If detection is successful, a "FATFS TEST" label is displayed in the center of the LCD, as shown below:



FATFS TEST

5.5.4 Program Analysis

Note

`lv_mainstart.c` is the file in the entire LVGL application project that most intuitively reflects the experimental effect. Therefore, this chapter focuses on analyzing the program in `lv_mainstart.c` to help directly parse the core functions in the project files.

- `lv_mainstart.c` file

1. `lv_tell()`: File read/write pointer position retrieval function. `lv_tell()` is a wrapper function for the LVGL file system native interface `lv_fs_tell`. It is used to get the offset of the file read/write pointer relative to the file header, simplifying the call to the native interface.

```
/**
 * @brief    Get read/write pointer position
 * @param    fd: File structure pointer
 * @return    Returns the read/write pointer position in the file (offset from header)
 */
long lv_tell(lv_fs_file_t *fd)
{
    uint32_t pos = 0;
    lv_fs_tell(fd, &pos);
    return pos;
}
```

2. `lv_fs_test()`: LVGL file system read/write test function. `lv_fs_test()` is a static function responsible for implementing file read/write tests based on LVGL file system interfaces. It includes two core processes: file writing and file reading, and prints test results via the serial port for debugging.

```
/**
 * @brief File system test
 * @param None
 * @return None
 */
static void lv_fs_test(void)
{
    char rbuf[30] = {0};    /* Buffer for data read from file */
    uint32_t wsize = 0;     /* Number of bytes written to file */
    uint32_t rsize = 0;     /* Number of bytes read from file */
    lv_fs_file_t fd;        /* File system structure variable */
    lv_fs_res_t res;        /* File operation return value */

    /* Write test (Note: filename cannot contain underscores) */
    res = lv_fs_open(&fd, "0:/lvgl.txt", LV_FS_MODE_WR);    /* Open file with write
permission; creates if not exists on SD card */
    if (res != LV_FS_RES_OK)
    {
        printf("open 0:/lvgl.txt ERROR\n");
        return ;
    }

    res = lv_fs_write(&fd, "hello_lvgl_123", 15, &wsize);    /* Write data to the
successfully opened file */
    if (res != LV_FS_RES_OK)
    {
        printf("write /lvgl.txt fail\n");
        return ;
    }
    printf("write data to file succeed \n");    /* Serial print indicating
successful data write */

    lv_fs_close(&fd);    /* Close file */

    /* Read test */
    res = lv_fs_open(&fd, "0:/lvgl.txt", LV_FS_MODE_RD);    /* Open file with read
permission */
    if (res != LV_FS_RES_OK)
    {
        printf("open 0:/lvgl.txt ERROR\n");
        return ;
    }

    lv_tell(&fd);    /* Get read/write pointer
position */
    lv_fs_seek(&fd, 0, LV_FS_SEEK_SET);    /* Set read/write pointer to
file header */
}
```

```

    lv_tell(&fd);                                /* Get read/write pointer
position */

    res = lv_fs_read(&fd, rbuf, 100, &rsz);      /* Read file data from the
successfully opened file */
    if (res != LV_FS_RES_OK)
    {
        printf("read ERROR\n");
        return ;
    }

    printf("file:lvgl.txt rd: %s \n", rbuf);      /* Serial print displaying read
data */

    lv_fs_close(&fd);                            /* Close file */
}

```

3. `lv_mainstart()`: LVGL demo entry function. `lv_mainstart()` is the entry function for the LVGL demonstration features. It creates a label widget on the LVGL graphical interface to display text and triggers the LVGL file system read/write test process.

```

/**
 * @brief LVGL Demo
 * @param None
 * @return None
 */
void lv_mainstart(void)
{
    lv_obj_t *label = lv_label_create(lv_scr_act()); /* Create label */
    lv_label_set_text(label, "FATFS TEST");          /* Display string */
    lv_obj_center(label);                            /* Center align */
    lv_fs_test();                                    /* File system test */
}

```

5.6 LVGL Built-in Font Application Experiment

5.6.1 Experiment Introduction

This experiment realizes custom font loading and text display functions based on the LVGL graphics library. It primarily verifies LVGL's ability to reference and render external custom fonts, as well as the basic configuration methods for interface styles and widget layouts. The experiment introduces a custom font file via the font declaration macro provided by LVGL, creates a label widget, configures the font, text, and background color, and finally displays the specified content on the main LVGL screen.

5.6.2 Program Download

Locate [Source Code](#)->**06_lvgl_font** under the same path as this document and click **Select Folder**.

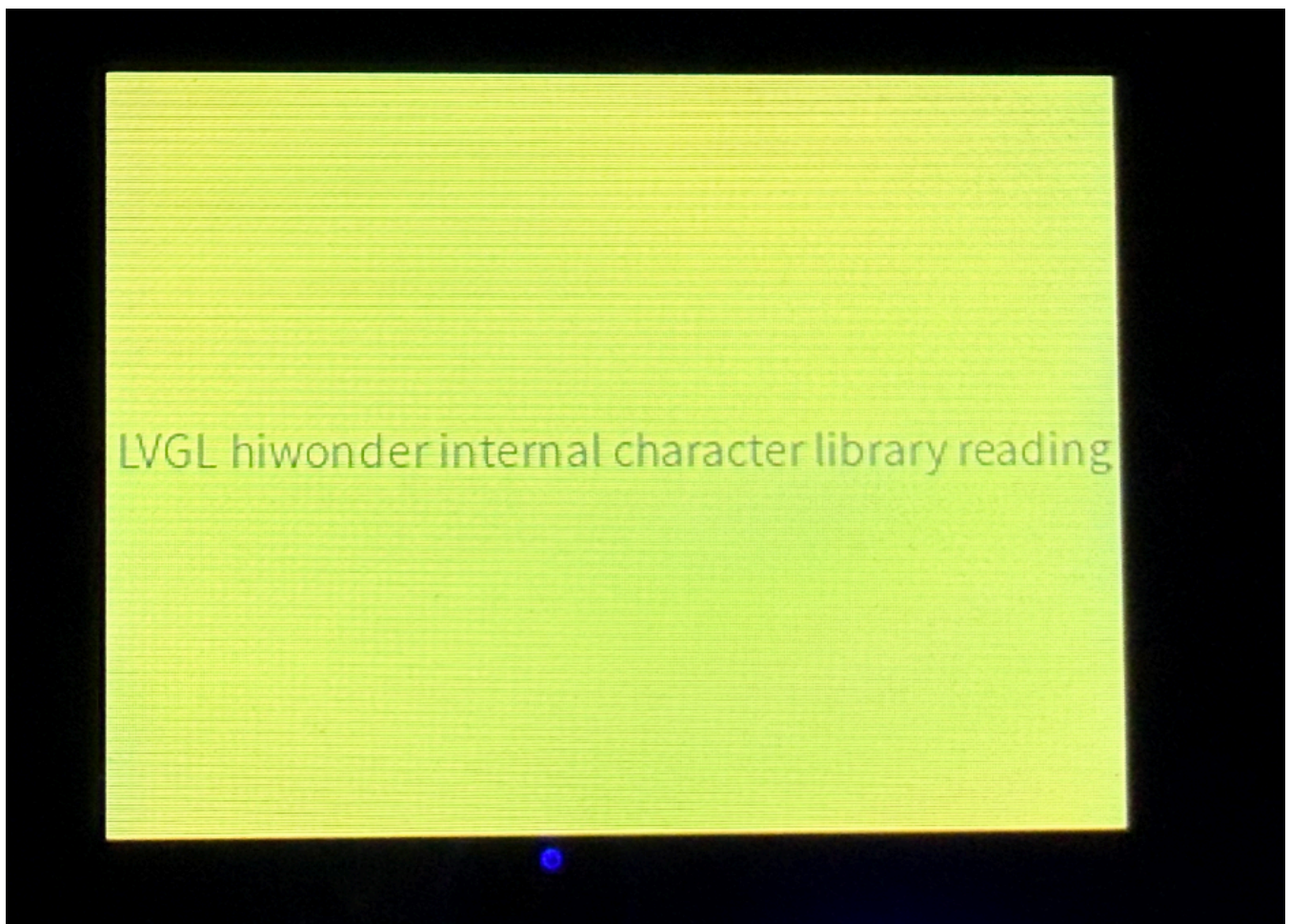
Note

Ensure the program file path contains only English characters, as non-English characters may cause compilation failures.

For specific program download tutorials, refer to [5.1 Program Download Instructions \(Must Read\)](#).

5.6.3 Program Outcome

The background color of the current active LVGL screen is set to yellow from the LVGL built-in palette, and the text `LVGL hiwonder internal character library reading` is displayed in the center of the screen, as shown below:



5.6.4 Program Analysis

Note

`lv_mainstart.c` is the file in the entire LVGL application project that most intuitively reflects the experimental effect. Therefore, this chapter focuses on analyzing the program in `lv_mainstart.c` to help directly parse the core functions in the project files.

- **lv_mainstart.c File Analysis**

1. **Font Declaration and Global Variable Definition:** The beginning of the program completes the declaration of the custom font and the global definition of the LVGL label object pointer, providing basic resources and object references for subsequent font display on the graphical interface. The `myFont14` custom font is declared using the `LV_FONT_DECLARE` macro, which is the font reference interface provided by LVGL. A global pointer `font_label` of type `lv_obj_t` is defined to point to the font display label object created later.

```
LV_FONT_DECLARE(myFont14)    /* Declare myFont14 font, myFont14.c needs to be placed in
"LVGL\src\font" folder */
lv_obj_t *font_label;        /* Define a font label pointer */
```

2. **lv_mainstart():** `lv_mainstart()` is the core entry function for the LVGL font demo. It is responsible for screen background settings, font label creation, custom font loading, text content setting, and label centering display. `lv_obj_set_style_bg_color` is called to set the background color of the current active screen `lv_scr_act()` in the default state `LV_STATE_DEFAULT`, using yellow from the LVGL built-in palette via `lv_palette_main(LV_PALETTE_YELLOW)`. `lv_obj_set_style_text_font` sets the display font of the created label in the default state to the previously declared `myFont14` custom font, implementing LVGL's reference and rendering of external font files. `lv_label_set_text` sets the text string to be displayed by the label to `LVGL hiwonder internal character library reading`. `lv_obj_center` aligns the label widget to the center of its parent object horizontally and vertically, ensuring the visual centering of the interface element.

```
/**
 * @brief LVGL Demo
 * @param None
 * @return None
 */
void lv_mainstart(void)
{
    lv_obj_set_style_bg_color(lv_scr_act(), lv_palette_main(LV_PALETTE_YELLOW),
LV_STATE_DEFAULT); /* Set background color */
    font_label = lv_label_create(lv_scr_act()); /* Create label
*/
    lv_obj_set_style_text_font(font_label, &myFont14, LV_STATE_DEFAULT); /* Set font */
    lv_label_set_text(font_label, "LVGL hiwonder internal character library reading");
        /* Set text */
    lv_obj_center(font_label); /* Center align
display */
}
```

5.7 LVGL External Font Loading and Display Experiment

5.7.1 Experiment Introduction

This experiment validates custom font loading and style customization based on the LVGL graphics library. The core objective is to test reading the pre-flashed Font24 font file from SPIFLASH and configuring text font and color attributes uniformly via the LVGL style system.

5.7.2 Program Download

Locate [Source Code](#)->**07_lvgl_spiflash_font** under the same path as this document and click **Select Folder**.

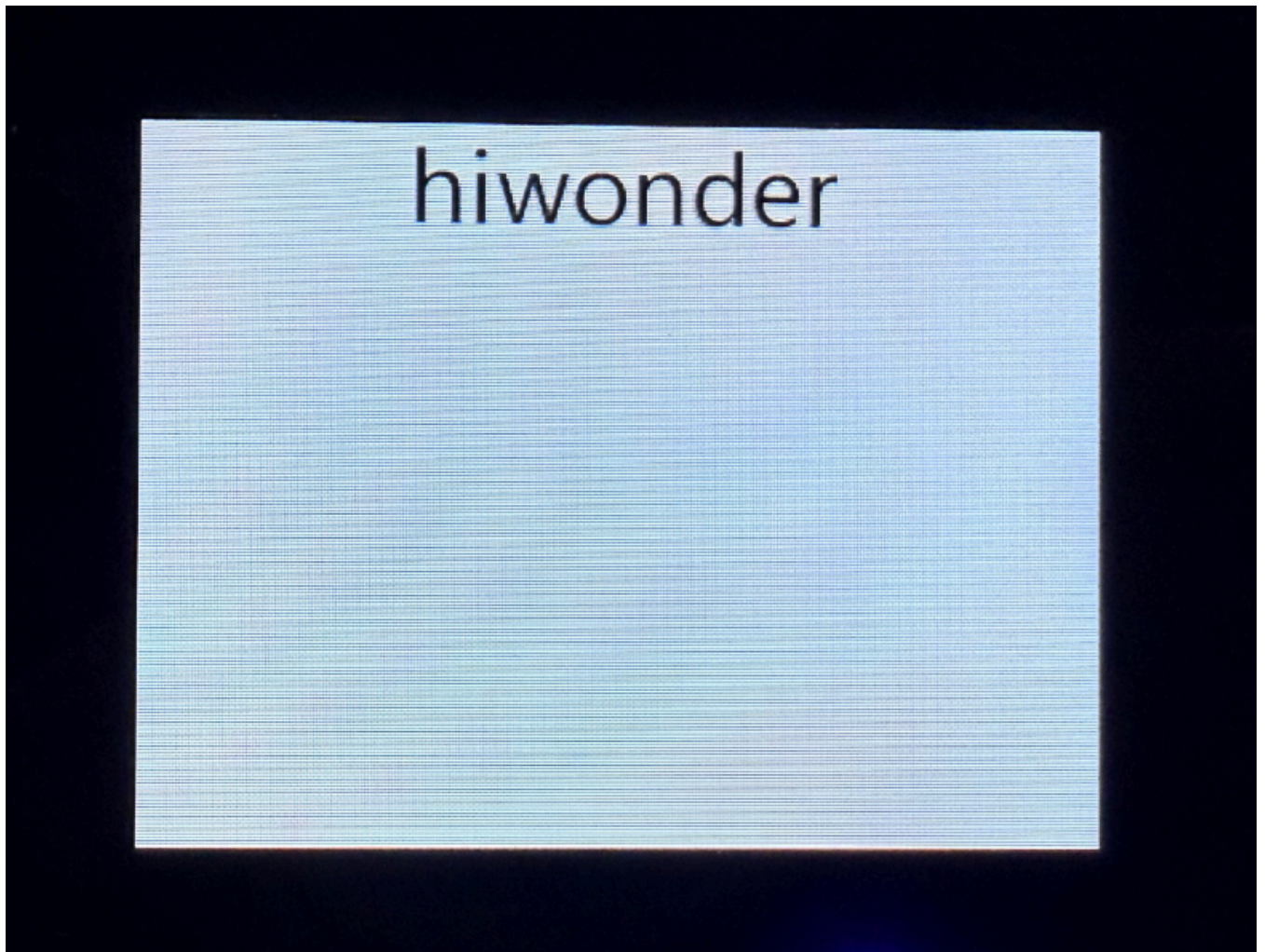
Note

Ensure the program file path contains only English characters, as non-English characters may cause compilation failures.

For specific program download tutorials, refer to [5.1 Program Download Instructions \(Must Read\)](#).

5.7.3 Program Outcome

After running the experiment, the text `hiwonder` is displayed in the top center of the current active LVGL screen. The text is rendered using the Font24 font loaded from SPIFLASH, with clear characters matching the Font24 definition, and the text color is pure black, as shown below:



5.7.4 Program Analysis

Note

`lv_mainstart.c` is the file in the entire LVGL application project that most intuitively reflects the experimental effect. Therefore, this chapter focuses on analyzing the program in `lv_mainstart.c` to help directly parse the core functions in the project files.

- **lv_mainstart.c File Analysis**

1. Font Declaration, Style, and Global Variable Definition: The `Font24` font is declared via the `LV_FONT_DECLARE` macro. This font has been flashed to SPIFLASH, allowing LVGL to load font data directly from hardware storage. A `static` style variable `font_style` of type `lv_style_t` is defined. `static` ensures it is visible only in the current file, used for unified management of label text font and color style attributes, enabling style reuse and centralized configuration. A global pointer `font_label` of type `lv_obj_t` is defined to point to the subsequently created text label object. The global scope allows it to be accessed and modified in other functions, supporting dynamic updates of the label.

```
LV_FONT_DECLARE(Font24)          /* Declare Font24, this font is already burned into SPIFLASH */
static lv_style_t font_style;      /* Define style variable */
lv_obj_t *font_label;             /* Define a font label pointer */
```

2. `lv_mainstart`: This function serves as the core entry point for LVGL style customization and text display. It is responsible for executing a sequence of interface initialization operations, including style initialization, text label creation, style binding, text content setting, and label layout alignment. The function calls `lv_style_init` to initialize the `font_style` variable. This step is a prerequisite for utilizing LVGL styles and ensures that style attributes are set to their default initial state. It then calls `lv_label_create` to generate a label object, specifying the current active screen obtained via `lv_scr_act` as the parent object, and assigns the object handle to the global `font_label` pointer. Next, `lv_obj_add_style` is invoked to apply the configured `font_style` to the label widget. The third parameter is set to 0 to indicate the default style priority, which allows the label to adopt the font and color attributes defined by that style. Subsequently, `lv_label_set_text` sets the label text to `hiwonder` to demonstrate the style content. Finally, `lv_obj_align` aligns the label to the top-middle position of the parent active screen using the `LV_ALIGN_TOP_MID` constant with zero offset on both axes. This alignment ensures the top of the label coincides with the top of the screen and is centered horizontally.

```
/**
 * @brief LVGL Demo
 * @param None
 * @return None
 */
void lv_mainstart(void)
{
    lv_style_init(&font_style);          /* Initialize style */
    lv_style_set_text_font(&font_style, &Font24); /* Set text font attribute */
    lv_style_set_text_color(&font_style, lv_color_black()); /* Set text color attribute */

    font_label = lv_label_create(lv_scr_act()); /* Create a label */
    lv_obj_add_style(font_label, &font_style, 0); /* Add style to font label */
    lv_label_set_text(font_label, "hiwonder"); /* Set text */
    lv_obj_align(font_label, LV_ALIGN_TOP_MID, 0, 0); /* Top middle alignment */
}
```

5.8 LVGL TrueType Font Support Experiment

5.8.1 Experiment Introduction

This experiment validates the loading and rendering capabilities of the LVGL graphics library for custom fonts in TTF format. The core objective is to test whether LVGL can correctly declare and load the size 36 font using the `LV_FONT_DECLARE` macro when the `myttfFont36.c` file, converted from an external TTF font, is placed in the LVGL source `src/font` directory. Simultaneously, the experiment utilizes LVGL's style configuration functions to set the screen background color, create a text label, and complete font binding and text display.

5.8.2 Program Download

Locate [Source Code](#)->**08_lvgl_ttf_font** under the same path as this document and click **Select Folder**.

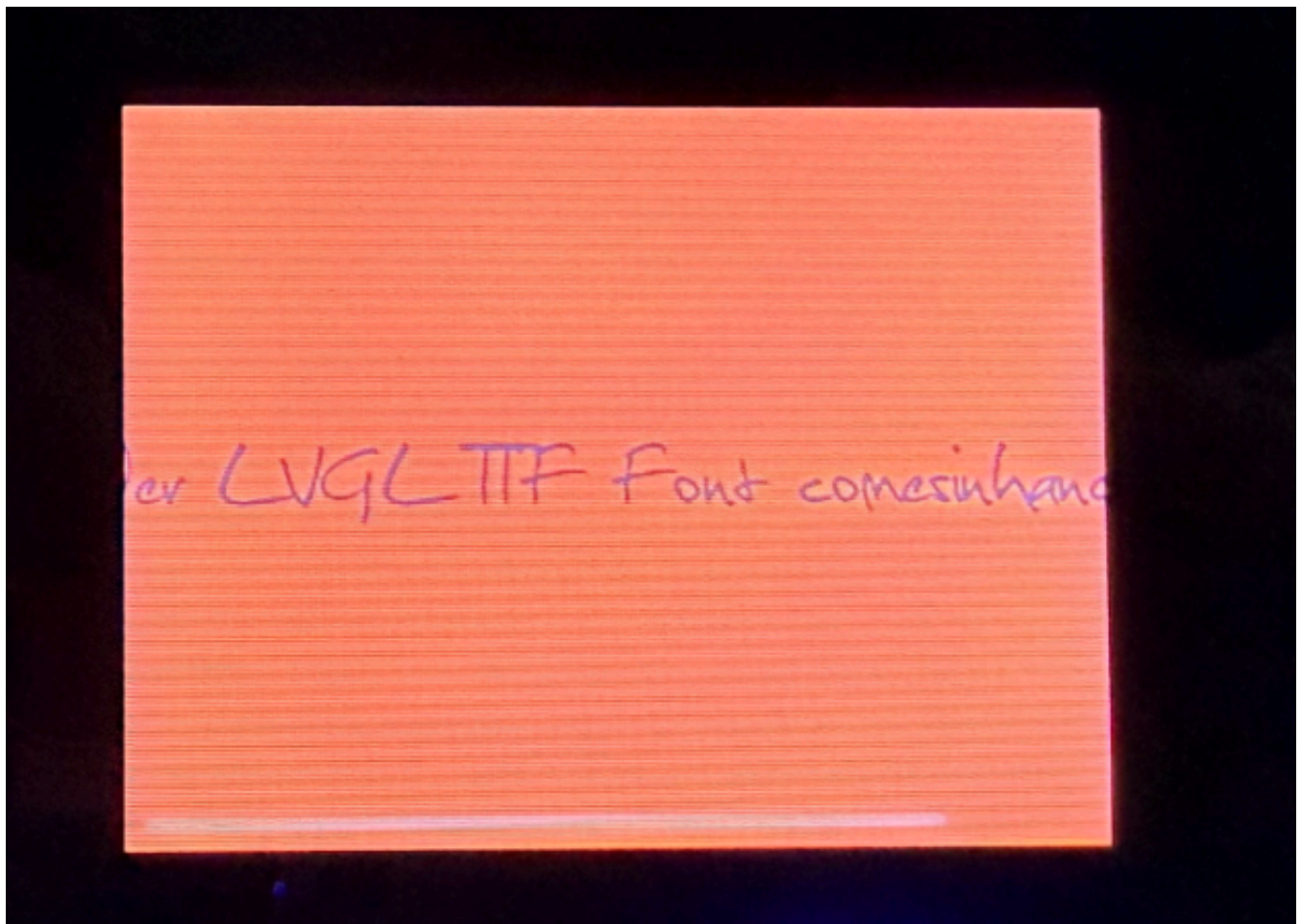
Note

Ensure the program file path contains only English characters, as non-English characters may cause compilation failures.

For specific program download tutorials, refer to [5.1 Program Download Instructions \(Must Read\)](#).

5.8.3 Program Outcome

After the program is successfully uploaded, the current active LVGL screen presents a pure red background from the LVGL built-in palette. The text "Hiwonder LVGL TTF Font comesinhandy.ttf!" is displayed in the exact center of the screen horizontally and vertically. The content can be scrolled by touching the screen, as shown below:



5.8.4 Program Analysis

Note

`lv_mainstart.c` is the file in the entire LVGL application project that most intuitively reflects the experimental effect. Therefore, this chapter focuses on analyzing the program in `lv_mainstart.c` to help directly parse the core functions in the project files.

• `lv_mainstart.c`

1. Font Declaration and Global Variable Definition: The `myttfFont36` TTF custom font is declared via the `LV_FONT_DECLARE` macro. This macro requires the corresponding font conversion file `myttfFont36.c` to be placed in the `LVGL\src\font` directory to ensure LVGL can correctly load and parse the TTF font data. A global pointer `font_label` of type `lv_obj_t` is defined to point to the created TTF font display label object. The global scope allows access and modification in other functions, facilitating subsequent operations like dynamic updates or style modifications.

```
LV_FONT_DECLARE(myttfFont36)    /* Declare myttfFont36 font, myttfFont36.c needs to be in  
"LVGL\src\font" folder */  
lv_obj_t *font_label;           /* Define a font label pointer */
```

2. `lv_mainstart`: This function serves as the core entry point for LVGL TTF font rendering and interface display. It is responsible for executing a series of interface initialization operations, including setting the screen background color, creating the TTF font label, configuring the font, setting the text content, and centering the label. The function calls `lv_obj_set_style_bg_color` to configure the background color of the current active LVGL screen for the default state, utilizing red from the built-in LVGL palette via the `lv_palette_main` function with the `LV_PALETTE_RED` constant. Subsequently, `lv_label_create` generates a label object using the current active screen as the parent. The object handle is assigned to the global `font_label` pointer, completing the instantiation of the label as the carrier for TTF font rendering. The function then uses `lv_obj_set_style_text_font` to set the default display font of the created label to the previously declared `myttfFont36` custom TTF font, thereby implementing the referencing and rendering of the TTF format font by LVGL. Following this, `lv_label_set_text` sets the label text string to `Hiwonder LVGL TTF Font comesinhandy.ttf!` to demonstrate the TTF font rendering effect. Finally, `lv_obj_center` aligns the label widget to the center of its parent active screen both horizontally and vertically, ensuring the text label is positioned in the visual center of the screen without offset or misalignment.

```
/**  
 * @brief LVGL Demo  
 * @param None  
 * @return None  
 */  
void lv_mainstart(void)  
{  
    lv_obj_set_style_bg_color(lv_scr_act(), lv_palette_main(LV_PALETTE_RED),  
LV_STATE_DEFAULT);    /* Set background color */  
    font_label = lv_label_create(lv_scr_act());  
    /* Create label */
```

```
lv_obj_set_style_text_font(font_label, &myttfFont36, LV_STATE_DEFAULT);
    /* Set font */
lv_label_set_text(font_label, "Hiwonder LVGL TTF Font comesinhandy.ttf!");
    /* set text */
lv_obj_center(font_label);
    /* center align display */
}
```

5.9 LVGL Basic Objects and Widgets Experiment

5.9.1 Experiment Introduction

This experiment realizes basic object creation, style configuration, and event response functions based on the LVGL graphics library. By creating parent and child rectangle objects on the screen, binding a long-press event to the parent object and a click event to the child object, the position of the corresponding objects is dynamically adjusted in the event callback function based on the trigger source.

5.9.2 Program Download

Locate [Source Code](#)->09_lvgl_obj under the same path as this document and click **Select Folder**.

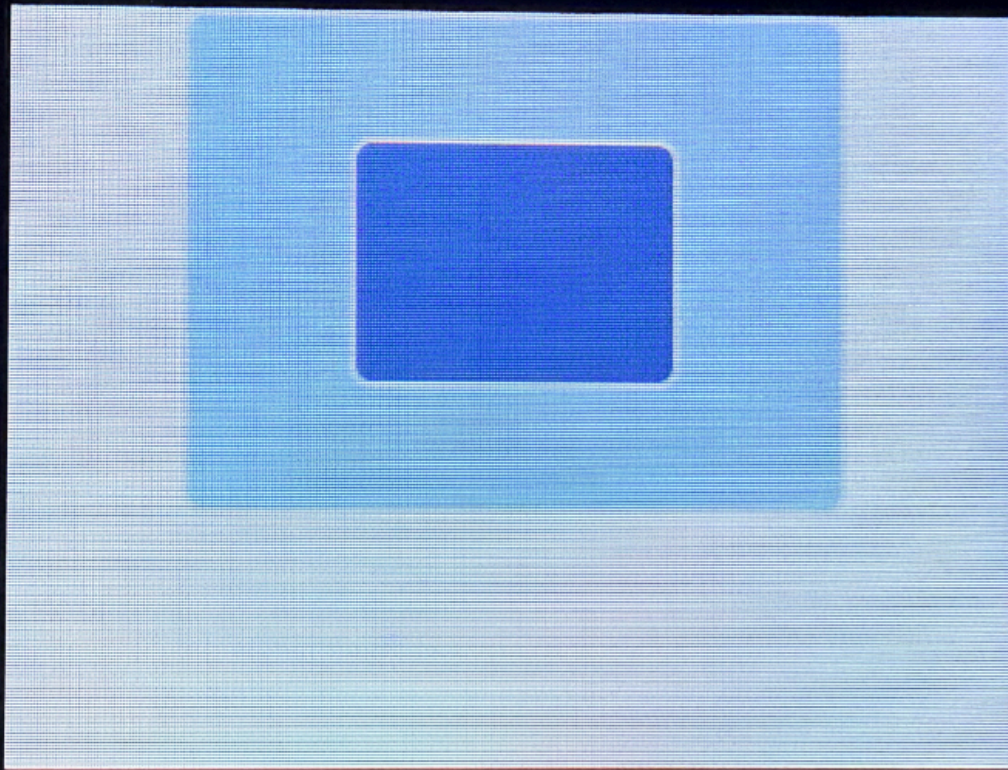
Note

Ensure the program file path contains only English characters, as non-English characters may cause compilation failures.

For specific program download tutorials, refer to [5.1 Program Download Instructions \(Must Read\)](#).

5.9.3 Program Outcome

After the program runs, the screen displays a light blue parent object occupying 2/3 of the screen size, containing a dark blue child object occupying 1/3 of the screen size. When the light blue parent object is long-pressed, it moves immediately to the center of the screen. When the dark blue child object is clicked and released, it adjusts to a position centered on the right side of itself with an X-axis offset of 100. The background color and size of the parent and child objects remain unchanged during operation, as shown below:



5.9.4 Program Analysis

Note

`lv_mainstart.c` is the file in the entire LVGL application project that most intuitively reflects the experimental effect. Therefore, this chapter focuses on analyzing the program in `lv_mainstart.c` to help directly parse the core functions in the project files.

- **lv_mainstart.c**

1. Static Variables and Object Pointer Definition: `scr_act_width` and `scr_act_height` are static variables of type `lv_coord_t`, used to store the dynamically retrieved width and height of the current active screen. `parent_obj` and `child_obj` are pointers of type `lv_obj_t`, pointing to the parent container object and child widget object created later. Global pointers allow direct access and modification of object attributes in event callback functions.

```
static lv_coord_t scr_act_width, scr_act_height;    /* Active screen width, height */
static lv_obj_t *parent_obj;                       /* Define parent object */
static lv_obj_t *child_obj;                        /* Define child object */
```

2. `obj_event_cb`: This static event callback function is responsible for handling trigger events for both parent and child objects. It dynamically adjusts the display position of the corresponding object based on the event source, serving as the core implementation of the LVGL event-driven mechanism. The function calls `lv_event_get_target(e)` to extract the object initiating the event from the event parameter

collection `e` and assigns it to the `target` pointer. If the trigger source is the `parent_obj`, the function calls `lv_obj_align` to realign the parent object to the exact center of the screen using the `LV_ALIGN_CENTER` constant with zero offset on both the X and Y axes, thereby resetting the parent object's position. If the trigger source is the `child_obj`, `lv_obj_align` is invoked to align the child object to the right-middle position relative to its parent reference point using `LV_ALIGN_RIGHT_MID`, accompanied by a 100-pixel offset along the X-axis to achieve dynamic position adjustment.

```
/**
 * @brief Basic object event callback
 * @param *e : Collection of event-related parameters
 * @return None
 */
static void obj_event_cb(lv_event_t *e)
{
    lv_obj_t * target = lv_event_get_target(e);           /* Get event trigger source */

    if (target == parent_obj)                             /* Trigger source: is it
parent object? */
    {
        lv_obj_align(parent_obj, LV_ALIGN_CENTER, 0, 0); /* Reset parent object
position: Center */
    }
    else if (target == child_obj)                         /* Trigger source: is it
child object? */
    {
        lv_obj_align(child_obj, LV_ALIGN_RIGHT_MID, 100, 0); /* Reset child object
position: Right Middle, offset 100 on x axis */
    }
}
```

3. `lv_mainstart()`: The core entry function for LVGL object creation, style configuration, and event binding. It dynamically retrieves screen dimensions, creates parent and child objects, customizes styles, and binds event callbacks. It first uses `lv_obj_get_width(lv_scr_act())` and `lv_obj_get_height(lv_scr_act())` to get and store screen dimensions for adaptive sizing. It creates a parent object using the active screen as the container, sets its size to 2/3 of the screen, initial position to top center, background color to light blue (0x99ccff), and binds the `obj_event_cb` callback to respond to long-press events. Then, it creates a child object within the parent, sets its size to 1/3 of the screen, initial position to the center of the parent, background color to dark blue (0x003366), and binds `obj_event_cb` to respond to click-release events. This implementation realizes the hierarchical nesting of LVGL objects and simultaneously achieves interaction design between parent and child objects through the binding of distinct events.

```
/**
 * @brief LVGL Demo
 * @param None
 * @return None
 */
void lv_mainstart(void)
{
```

```

    /* Dynamically get screen size */
    scr_act_width = lv_obj_get_width(lv_scr_act());           /* Get
active screen width */
    scr_act_height = lv_obj_get_height(lv_scr_act());         /* Get
active screen height */

    /* Parent object */
    parent_obj = lv_obj_create(lv_scr_act());                 /* Create
parent object */
    lv_obj_set_size(parent_obj, scr_act_width * 2/3, scr_act_height * 2/3); /* Set
parent object size */
    lv_obj_align(parent_obj, LV_ALIGN_TOP_MID, 0, 0);          /* Set
parent position: Top Middle */
    lv_obj_set_style_bg_color(parent_obj, lv_color_hex(0x99ccff), 0); /* Set
parent bg color: Light Blue */
    lv_obj_add_event_cb(parent_obj, obj_event_cb, LV_EVENT_LONG_PRESSED, NULL); /* Add
event: Long press triggers */

    /* Child object */
    child_obj = lv_obj_create(parent_obj);                     /* Create
child object */
    lv_obj_set_size(child_obj, scr_act_width / 3, scr_act_height / 3); /* Set child
object size */
    lv_obj_align(child_obj, LV_ALIGN_CENTER, 0, 0);            /* Set child
position: Center */
    lv_obj_set_style_bg_color(child_obj, lv_color_hex(0x003366), 0); /* Set child
bg color: Dark Blue */
    lv_obj_add_event_cb(child_obj, obj_event_cb, LV_EVENT_CLICKED, NULL); /* Add
event: Click release triggers */
}

```

5.10 LVGL Arc Widget Application Experiment

5.10.1 Experiment Introduction

This experiment realizes a double-arc linkage and a real-time percentage label update function based on the LVGL graphics library. The program adapts font size and arc width based on the active screen width. It creates an interactive left arc and a display-only right arc, binds a value change event to the left arc, verifying the implementation logic of style adaptation, event response, and widget linkage for the LVGL Arc component.

5.10.2 Program Download

Locate [Source Code](#)->10_lvgl_arc under the same path as this document and click **Select Folder**.

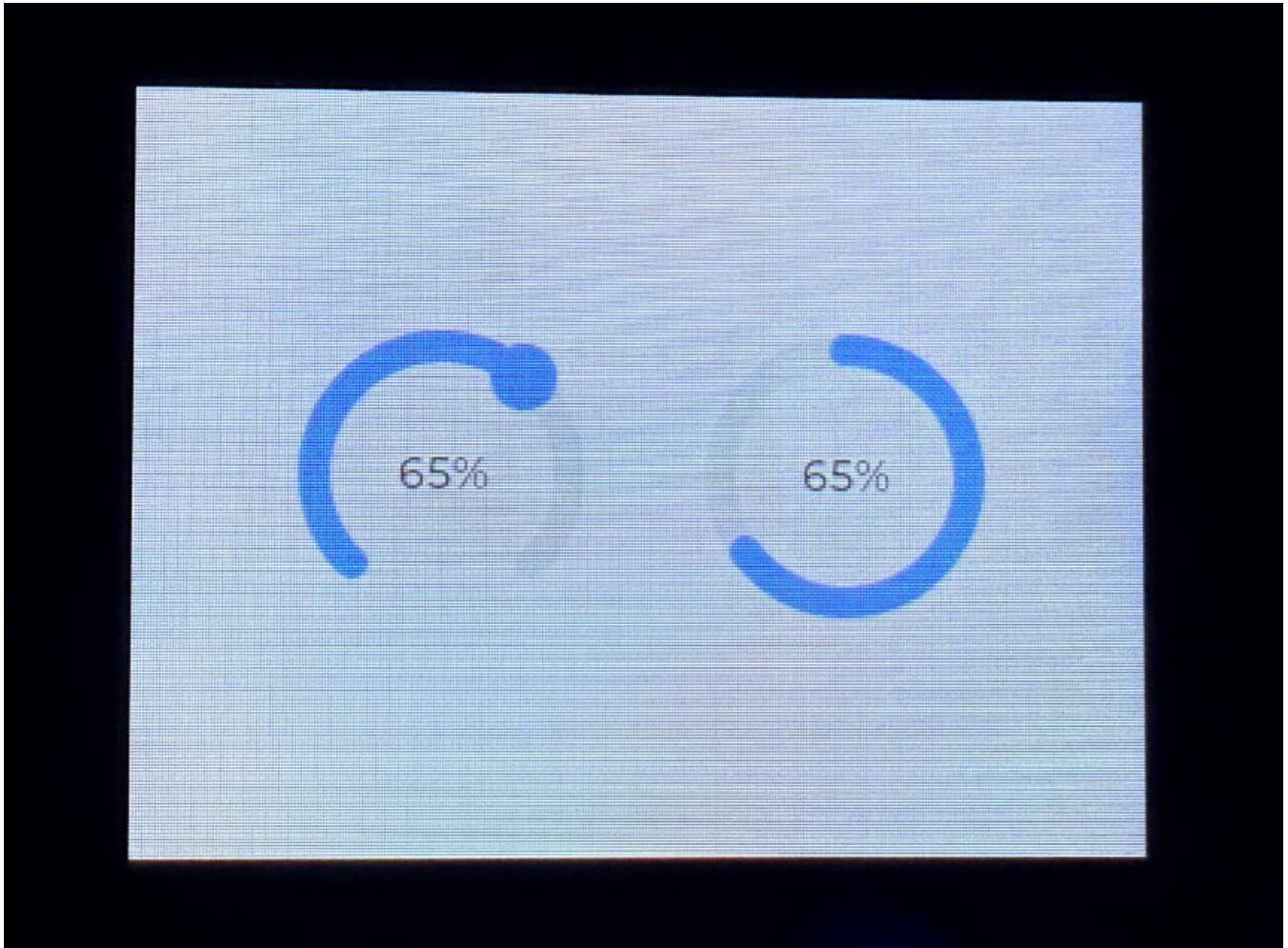
Note

Ensure the program file path contains only English characters, as non-English characters may cause compilation failures.

For specific program download tutorials, refer to [5.1 Program Download Instructions \(Must Read\)](#).

5.10.3 Program Outcome

After the program runs, an arc component is displayed on the left and right sides of the screen center. Initially, both arcs show 0%, and the corresponding percentage labels display 0%. Dragging the knob of the left arc updates the percentage labels on both sides in real-time, and the value of the right arc synchronizes with the left arc. The right arc has no knob and cannot be interacted with manually. Furthermore, the program automatically adapts font size and arc width based on screen width, maintaining a centered left-right layout, as shown below:



5.10.4 Program Analysis

Note

`lv_mainstart.c` is the file in the entire LVGL application project that most intuitively reflects the experimental effect. Therefore, this chapter focuses on analyzing the program in `lv_mainstart.c` to help directly parse the core functions in the project files.

• `lv_mainstart.c`

1. Macro Definitions and Global Variable Definitions: The macros `scr_act_width` and `scr_act_height` encapsulate the interfaces used to retrieve the width and height of the active screen, thereby simplifying the method for accessing screen dimensions. Furthermore, pointers of type `lv_obj_t` are defined to reference the left and right percentage labels and arcs, alongside a variable of type `const lv_font_t*` for the font and a variable of type `uint8_t` to store the arc width.

```

/* Get current active screen width and height */
#define scr_act_width() lv_obj_get_width(lv_scr_act())
#define scr_act_height() lv_obj_get_height(lv_scr_act())

static lv_obj_t *label_left;          /* Left percentage label */
static lv_obj_t *label_right;         /* Right percentage label */
static lv_obj_t *arc_left;            /* Left arc */
static lv_obj_t *arc_right;           /* Right arc */

static const lv_font_t *font;         /* Define font */
static uint8_t arc_width;             /* Arc width */

```

2. `arc_event_cb`: `arc_event_cb` acts as a static event callback function responsible for handling arc value change events. It facilitates the synchronization between arc values and label displays, as well as the linkage between the left and right arcs. The function begins by retrieving the event trigger source object and the event type using `lv_event_get_target` and `lv_event_get_code`, respectively. Upon detecting that the event type is `LV_EVENT_VALUE_CHANGED`, indicating a value change, it calls `lv_arc_get_value` to obtain the current value of the triggering arc. Subsequently, `lv_label_set_text_fmt` is utilized to update the display text of both the left and right percentage labels to the corresponding percentage format. Simultaneously, `lv_arc_set_value` is invoked to synchronize the current value of the left arc to the right arc. This process achieves real-time synchronization between arc value changes and label displays, along with the numerical linkage effect between the two arcs.

```

/**
 * @brief Arc event callback
 * @param *e : Collection of event-related parameters
 * @return None
 */
static void arc_event_cb(lv_event_t *e)
{
    lv_obj_t *target = lv_event_get_target(e);          /* Get
trigger source */
    lv_event_code_t code = lv_event_get_code(e);        /* Get event
type */

    if(code == LV_EVENT_VALUE_CHANGED)
    {
        lv_label_set_text_fmt(label_left, "%d%%", lv_arc_get_value(target)); /* Get
current value, update left label */
        lv_label_set_text_fmt(label_right, "%d%%", lv_arc_get_value(target)); /* Get
current value, update right label */
        lv_arc_set_value(arc_right, lv_arc_get_value(target)); /* Get left
arc value, update right arc value */
    }
}

```

3. `lv_example_arc1`: This function is responsible for the creation, style configuration, and event binding of the left interactive arc and its corresponding percentage label, serving as the core implementation for the visualization and interaction of the left arc. The function first evaluates the active screen width obtained via the `scr_act_width` macro. If the width is less than or equal to 480, the `lv_font_montserrat_14` font is selected, and the arc width is set to 10; otherwise, the `lv_font_montserrat_30` font is selected with an arc width of 20, thereby achieving screen adaptation for both font and arc width. Subsequently, `lv_arc_create` is called to instantiate the left arc object using the current active screen as the parent container. Its dimensions are set to three-eighths of the screen height via `lv_obj_set_size`. `lv_obj_align` positions the arc at the screen center with a leftward offset equal to one-fifth of the screen width by `scr_act_width()/5`. The initial value is set to 0, and the widths of both the background arc and the foreground indicator arc are configured to the adapted `arc_width`. Simultaneously, the `arc_event_cb` callback is bound to the object to respond to value change events. Finally, the left percentage label is created and aligned to the same position as the left arc. Its initial text is set to "0%", and the adaptively selected font style is applied, completing the entire configuration for the left interactive arc and its corresponding label.

```
/**
 * @brief Left arc
 * @param None
 * @return None
 */
void lv_example_arc1(void)
{
    /* Select font based on active screen width */
    if (scr_act_width() <= 480)
    {
        font = &lv_font_montserrat_14;
        arc_width = 10;
    }
    else
    {
        font = &lv_font_montserrat_30;
        arc_width = 20;
    }

    /* Left arc */
    arc_left = lv_arc_create(lv_scr_act()); /* Create
arc */
    lv_obj_set_size(arc_left, scr_act_height() * 3/8, scr_act_height() * 3/8); /* Set size
*/
    lv_obj_align(arc_left, LV_ALIGN_CENTER, -scr_act_width()/5, 0); /* Set
position */
    lv_arc_set_value(arc_left, 0); /* Set
current value */
    lv_obj_set_style_arc_width(arc_left, arc_width, LV_PART_MAIN); /* Set
background arc width */
    lv_obj_set_style_arc_width(arc_left, arc_width, LV_PART_INDICATOR); /* Set
indicator arc width */
    lv_obj_add_event_cb(arc_left, arc_event_cb, LV_EVENT_VALUE_CHANGED, NULL); /* Add event
*/
}
```

```

    /* Left percentage label */
    label_left = lv_label_create(lv_scr_act());
percentage label */
    lv_obj_align(label_left, LV_ALIGN_CENTER, -scr_act_width()/5, 0);
position */
    lv_label_set_text(label_left, "0%");
*/
    lv_obj_set_style_text_font(label_left, font, LV_STATE_DEFAULT);
*/
}

```

4. `lv_example_arc2`: This function is responsible for the creation and style configuration of the static right arc and its corresponding percentage label, forming a linked display effect with the left arc. The function first calls `lv_arc_create` to instantiate the right arc object, using the current active screen as the parent container. Its dimensions are configured to three-eighths of the screen height, and its position is set to the screen center with a rightward offset equal to one-fifth of the screen width by `scr_act_width()/5`, with an initial value of 0. Next, the background arc angles are set to range from 0 to 360 degrees, and the rotation angle is set to 270 degrees. The function removes the knob style using `lv_obj_remove_style` and clears the clickable flag via `lv_obj_clear_flag`, transforming it into a static arc solely for display purposes. Simultaneously, the widths of both the background arc and the foreground indicator arc are set to the previously adapted `arc_width`. Subsequently, the right percentage label is created and aligned to the same position as the right arc. Its initial text is set to "0%", and the uniformly selected font style is applied. This completes the configuration of the right static arc and its corresponding label, enabling it to display values in linkage with the interactive left arc.

```

/**
 * @brief Right arc
 * @param None
 * @return None
 */
void lv_example_arc2(void)
{
    /* Right arc */
    arc_right = lv_arc_create(lv_scr_act());
arc */
    lv_obj_set_size(arc_right, scr_act_height() * 3/8, scr_act_height() * 3/8);
*/
    lv_obj_align(arc_right, LV_ALIGN_CENTER, scr_act_width()/5, 0);
position */
    lv_arc_set_value(arc_right, 0);
current value */
    lv_arc_set_bg_angles(arc_right, 0, 360);
background arc angles */
    lv_arc_set_rotation(arc_right, 270);
rotation angle */
    lv_obj_remove_style(arc_right, NULL, LV_PART_KNOB);
knob */
    lv_obj_clear_flag(arc_right, LV_OBJ_FLAG_CLICKABLE);
clickable attribute */
}

```

```

    lv_obj_set_style_arc_width(arc_right, arc_width, LV_PART_MAIN);           /* Set
background arc width */
    lv_obj_set_style_arc_width(arc_right, arc_width, LV_PART_INDICATOR);     /* Set
indicator arc width */

    /* Right percentage label */
    label_right = lv_label_create(lv_scr_act());                             /* Create
percentage label */
    lv_obj_align(label_right, LV_ALIGN_CENTER, scr_act_width()/5, 0);        /* Set
position */
    lv_label_set_text(label_right, "0%");                                     /* Set text
*/
    lv_obj_set_style_text_font(label_right, font, LV_STATE_DEFAULT);         /* Set font
*/
}

```

5. `lv_mainstart`: This function serves as the general entry point for the LVGL arc demonstration features, completing the initialization of the entire demonstration by invoking the configuration functions for the left and right arcs. Acting as the entry point, it only sequentially calls two functions: `lv_example_arc1` and `lv_example_arc2`. These functions respectively complete the creation, style configuration, event binding, and corresponding percentage label initialization for the interactive left arc and the static right arc. As the master control entry for the entire demonstration, it connects the full implementation logic of both arcs through simple function calls, thereby finalizing the overall initialization of the linked arc demonstration.

```

/**
 * @brief LVGL Demo
 * @param None
 * @return None
 */
void lv_mainstart(void)
{
    lv_example_arc1();               /* Left arc */
    lv_example_arc2();               /* Right arc */
}

```

Here is the English translation of the provided LVGL development tutorial.

5.11 LVGL Embedded Bootloader Design Experiment

5.11.1 Experiment Introduction

This experiment constructs a comprehensive application system integrating human-machine interaction, sensor data acquisition, multimedia processing, and system management. Upon power-up, the system sequentially completes the initialization of NVS storage, dual I2C buses, SPI bus, and various peripheral modules. These peripherals include the XL9555 IO expander, AHT30 temperature and humidity sensor, AP3216C optical sensor, QMI8658 six-axis inertial sensor, EEPROM, ES8311 audio codec, RTC real-time clock, touch screen, and LCD display module. Following hardware initialization, the program executes an onboard function self-check to verify the operational status of each peripheral module. Subsequently, the LVGL graphical interface demonstration program is launched to enable touch-screen-based graphical human-machine interaction.

5.11.2 Program Download

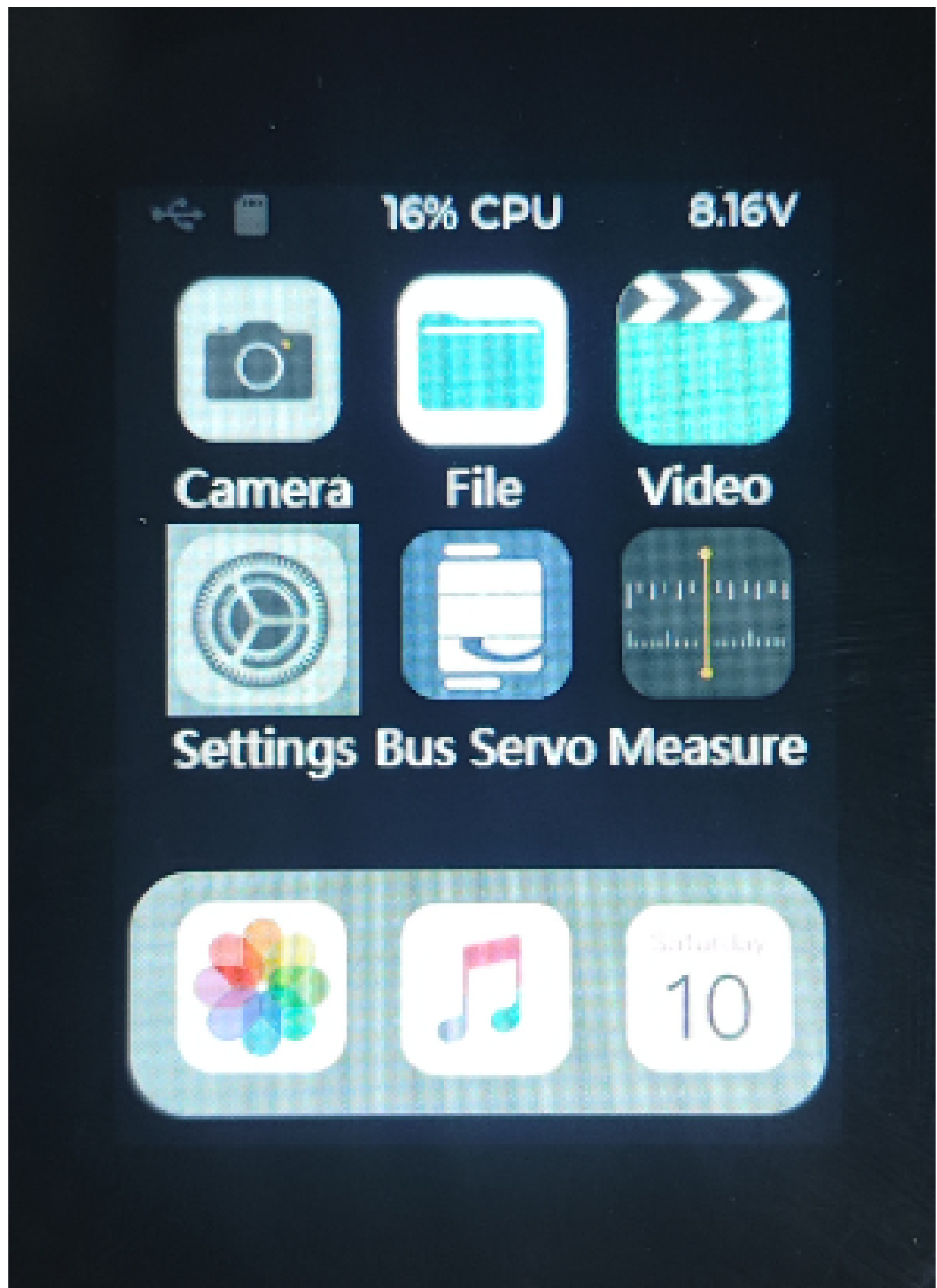
Locate the [Source Code](#) -> **11_bootloader** directory under the same path as this document and click **Select Folder**.

Note

Ensure the program file path contains only English characters, as non-English characters may cause compilation failures.

For specific Program Download Instructions (Must Read), refer to [5.1 Program Download Instructions \(Must Read\)](#).

5.11.3 Program Outcome



5.11.4 Program Analysis

Note

Since this experiment implements numerous functions and the program is complex, individual functions implementing specific features within the project are summarized and introduced below to provide a more intuitive understanding of the experiment's functionality. Therefore, this section does not analyze specific low-level functions line-by-line. The program files mentioned below are located in the directory [Source Code\11_bootloader\main\app_DEMO](#).

- **app_ui.c:** This is the core management program for the embedded application main interface, developed based on the LVGL graphics library. It primarily implements the initialization and layout of the device's main interface, including batch loading icons for camera, files, video, and other apps from SPI FLASH with automatic layout arrangement. It also monitors system status, including RTC time, battery voltage, CPU usage, and SD card insertion status, via timers and displays them visually in the top status bar. The program handles the automatic mounting and unmounting of the file system upon SD card insertion/removal. Additionally, it provides functions for creating and destroying custom-styled message boxes and manages icon memory during switching between app interfaces and the main interface.
- **image.c:** This is the image library management core based on the ESP32 platform. It implements the initialization and verification of the image library in the SPIFFS storage partition, data reading/writing, and sector erasure operations. It supports updating BIN files for 10 types of images, such as camera, files, video, etc., from an SD card or FLASH disk to the storage partition. Before updating, it checks for the existence of the target image file and erases partition sectors as needed to improve write efficiency. During the update, the LCD displays the erasure and writing progress percentage in real-time. Single image files are read and written in 4K blocks with 4KB address alignment. Upon completion, image library information is written to a specified partition address. Initialization involves multiple read verifications of this flag to determine if the image library is intact. If verification fails, an update prompt is issued.
- **lv_camera_demo.c:** This module demonstrates camera functionality using LVGL and the ESP32 platform. It handles camera initialization, real-time image acquisition and display, fill light control, and safe resource release. The `lv_camera_demo` function serves as the entry point: it initializes the camera, popping up a message on failure, hides the main interface, creates a camera-specific interface container, layouts the image display widget and fill light control button, binds click events, and creates a high-priority task `task1` pinned to a specific core. `task1` cyclically retrieves frame buffer data, handles pixel endianness conversion, allocates memory for converted data, displays the camera feed via an LVGL image widget, and uses a mutex semaphore to ensure thread safety for GUI operations. The fill light control toggles a status flag via button clicks, calls `camera_led_button_update` to update button text, and controls the `CAM_LED_IO` pin level via the XL9555 chip. The `lv_camera_de1` function handles resource cleanup, including deleting the camera task, freeing image buffers, de-initializing the camera, deleting the LVGL interface container, and restoring the main interface, while handling potential duplicate operations to ensure stability.
- **lv_clock_demo.c:** An analog clock demonstration module based on LVGL. It constructs a visual clock interface by loading dial background and hour/minute/second hand images. `lv_clock_demo` initializes the module by hiding the main interface, obtaining global RTC time data, creating a black background clock container, and positioning the dial and hands. It calculates initial rotation angles based on RTC time, adds the clock container to the key group for focus, and creates a timer with a 10ms period bound to the `lv_meter_timer` callback. `lv_meter_timer` reads the global RTC time in real-time, dynamically calculates rotation angles for the hands, and updates their display to simulate movement. `lv_clock_de1` cleans up

resources by deleting the timer and clock interface container, resetting global pointers, and restoring the main interface.

- **lv_file_demo.c:** A TF card (SD card) file browser demonstration module based on LVGL. It implements SD card initialization detection, prompting on failure, hides the main interface upon success, and creates a dedicated file browsing interface containing a title bar, file list area, and Back/Menu buttons. The program scans files and folders in a specified path, displaying them via an LVGL list widget with distinct icons for folders and different file types. It supports hierarchical folder access and path concatenation, managing directory navigation via return flags. It can read .txt file content for text display and load image files for display via LVGL image widgets. Interface buttons are bound to events for returning to the previous directory, returning to the root directory, or exiting the browser to restore the main interface.
- **lv_measure_demo.c:** A multi-sensor measurement demonstration module based on LVGL. It features two switchable display pages: environmental data and IMU data. Interaction via left/right arrow buttons switches pages and dynamically updates the title. The environmental data page uses a timer to collect and digitally display temperature sensor data and AP3216C distance/light data. The IMU data page uses a timer to read QMI8658 accelerometer and gyroscope raw data, updating values in a two-column, three-row layout. The program includes sensor initialization, widget style configuration, layout management, key group focus control, and resource cleanup via `lv_measure_del`, which handles duplicate click prevention to ensure stable operation and resource release upon exit.
- **lv_music_demo.c:** A WAV music player demonstration module based on LVGL and the ESP32 platform. It performs SD card detection, MUSIC folder verification, and WAV file parsing, handling audio output initialization and control via the ES8311 codec. The interface includes album art, song title labels, a playback progress bar, and Play/Pause/Prev/Next buttons bound to event callbacks. Two FreeRTOS tasks collaborate: the `music` task manages the playlist, parses audio file info, handles playback commands, and calculates duration/progress. The `play` task reads WAV audio data from the SD card and writes it to the audio output via ES8311. The program supports loop playback, real-time progress bar/time label updates, and handles exceptions like SD card removal or memory allocation failures. `lv_music_del` completes audio device de-initialization, task deletion, file/memory resource release, and restores the main interface.
- **lv_pic_demo.c:** An image browser demonstration module based on LVGL and ESP32. It detects the SD card, verifies the PICTURE folder, and counts BMP/JPG/JPEG/PNG files, using dynamic memory to store file information and index tables. The interface features a title bar, image display area, scrolling filename label, and Prev/Next buttons for image switching. A FreeRTOS `pic` task scans the folder to build an image index, reads the file corresponding to the current index, calls the appropriate decoding function, and displays the decoded data via an LVGL image widget. It implements cyclic switching logic, jumping to the end/beginning when reaching limits. The program handles exceptions with prompts, such as SD card removal, allocation failure, and empty folders. `lv_pic_del` handles task deletion, memory release, interface destruction, and the restoration of the main interface display, thereby ensuring the integrity of the image browsing function and the rationality of resource management.
- **lv_servo_demo.c:** This program serves as a bus servo debugging demonstration module implemented using the LVGL graphics library on the ESP32 platform. Its core functionality involves initializing UART0 to establish a communication link with the bus servos. The module constructs a visual debugging interface containing a position adjustment slider and a display area for nine servo parameters, which include the ID, current position, deviation, and position range. A timer periodically reads various servo operating parameters to update interface labels in real-time; should communication fail, the system displays placeholders and adjusts the refresh cycle. The slider is bound to value change events to dynamically set the target position of the servo and transmit movement commands. Furthermore, control

functions are provided to toggle the servo lock state between powered on and powered off modes, as well as to center the servo by moving it to position 500. The program incorporates object validity checks to prevent null pointer access. Upon exit, the `lv_servo_deinit` function performs resource cleanup by deleting the timer, destroying the interface container, resetting global objects, and restoring the main interface, thus ensuring stable operation and proper resource management.

- **lv_setting_demo.c:** This module demonstrates the system settings interface implemented using the LVGL graphics library on the ESP32 platform. It primarily constructs a settings interface container to display system information, including the SD card connection status, firmware version, ESP32 core count, and available heap memory, while also providing a return button prompt. A timer with a one-second cycle is employed to update the SD card status and available heap memory values in real-time. This timer includes multi-layer object validity checks to avoid accessing interface elements that have been deleted. The `lv_setting_deinit` function manages the interface exit logic by marking the interface as inactive, deleting the timer, cleaning up and destroying interface objects, resetting global pointers, and restoring the main interface. Furthermore, the settings interface is added to the key group to ensure button navigation functionality, thereby achieving the visual presentation and real-time updating of system information alongside safe resource management upon exit.
- **lv_video_demo.c:** This module demonstrates an AVI video player implemented using the LVGL graphics library on the ESP32 platform. Its core functionality involves detecting the SD card and scanning or counting MJPG-encoded AVI video files located in the VIDEO directory. To minimize fragmentation, the system prioritizes allocating memory for frame buffers and file information storage from PSRAM and reuses these resources. The module constructs a visual playback interface featuring a fixed 240x160 display area, a playback progress bar, time labels, and control buttons for Play, Pause, Previous Track, and Next Track. Event callbacks are bound to these control buttons to manage playback state transitions and video switching. A high-priority FreeRTOS task handles AVI file format parsing, invokes the MJPEG decoder to decode video frames, controls the playback frame rate via a timer, and skips audio stream data. Decoded data is cached and updated to the LVGL image widget in a non-blocking manner to render the video, while playback progress is updated in real-time. The system features a robust exception handling mechanism to manage scenarios such as SD card removal, decoding timeouts or failures, and file opening errors. It supports video loop playback and automatically resets the decoder to continue playback in the event of a decoding error. Upon exit, the `lv_video_deinit` function safely releases resources, including timers, file handles, decoding memory, and frame buffers. It also deletes the video task and interface objects and restores the main interface, thereby achieving efficient management of AVI video parsing, decoding, playback control, and system resources.

5.12 LVGL Custom Drawing Experiment

5.12.1 Experiment Introduction

This experiment implements a simple hand-drawing function based on the LVGL graphics library and touch screen driver. It involves creating interactive widgets to configure brush parameters and using a timer to scan touch status in real-time. It implements multi-point thick line drawing for capacitive screens and single-point drawing for resistive screens. Additionally, Clear Screen (RST) and Return to Main Interface (Back) buttons are designed to verify the implementation logic of LVGL event response, widget layout, touch screen driver linkage, and drawing adaptation methods for different touch screen types.

5.12.2 Program Download

Locate the [Source Code](#) -> **12_lvgl_paint** directory under the same path as this document and click **Select Folder**.

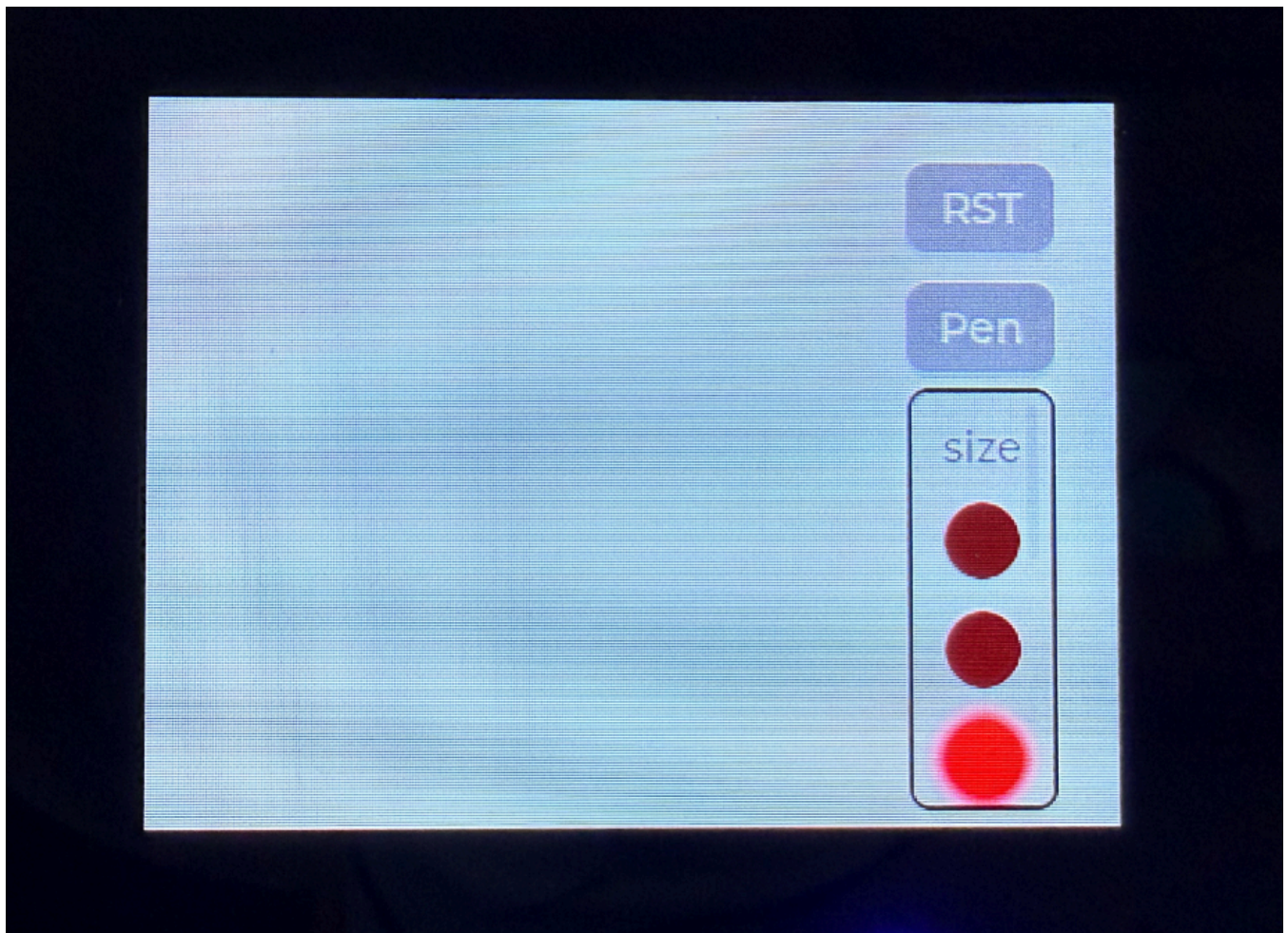
Note

Ensure the program file path contains only English characters, as non-English characters may cause compilation failures.

For specific Program Download Instructions (Must Read), refer to [5.1 Program Download Instructions \(Must Read\)](#).

5.12.3 Program Outcome

Upon running the program, the screen displays a white drawing area. The right side sequentially presents the RST (reset) label, brush size selection area, color selection area, and the Back button at the bottom. Clicking RST clears the drawing area to white. Selecting brush size/color lights up the corresponding LED and automatically switches the brush parameters. When touching the screen, a capacitive screen draws continuous thick lines based on the selected size and color. Clicking the Back button destroys the current drawing widgets and timer, resets brush parameters, and returns to the main interface. Throughout the process, a timer scans the touch status every 10ms to update the drawing content in real-time.



5.12.4 Program Analysis

Note

`lv_mainstart.c` is the file in the LVGL application project that most intuitively reflects the experiment's effects. Therefore, this chapter focuses on analyzing the program within `lv_mainstart.c` to help directly understand the core functions of the entire project.

- `lv_mainstart.c`

1. `lv_mainstart()`: The main initialization control function for the hand-drawing application. It is responsible for creating the drawing canvas, registering touch events, and loading the toolbar interface. It creates a full-screen drawing main container `lv_touch_cont`, sets it to a white background, and disables scrolling. It registers three core events, including Pressed, Pressing, and Released, for the container, binding them to the `touch_event_cb` callback function. Finally, it calls `lv_load_draw_dialog` to create the brush style adjustment toolbar.

```
void lv_mainstart(void)
{
    last_x = -1;
    last_y = -1;

    /* Create main container */
    lv_obj_t *scr = lv_scr_act();
    lv_coord_t scr_w = lv_obj_get_width(scr);
    lv_coord_t scr_h = lv_obj_get_height(scr);

    lv_touch_cont = lv_obj_create(scr);
    lv_obj_set_size(lv_touch_cont, scr_w, scr_h);
    lv_obj_clear_flag(lv_touch_cont, LV_OBJ_FLAG_SCROLLABLE);

    /* Set main container background to white (Canvas area) */
    lv_obj_set_style_bg_color(lv_touch_cont, lv_color_hex(0xFFFFFFFF), 0);
    lv_obj_set_style_bg_opa(lv_touch_cont, LV_OPA_COVER, 0);

    /* Ensure the main container is clickable to receive touch events */
    lv_obj_add_flag(lv_touch_cont, LV_OBJ_FLAG_CLICKABLE);

    /* Add touch events (Use line objects for drawing) */
    lv_obj_add_event_cb(lv_touch_cont, touch_event_cb, LV_EVENT_PRESSED, NULL);
    lv_obj_add_event_cb(lv_touch_cont, touch_event_cb, LV_EVENT_PRESSING, NULL);
    lv_obj_add_event_cb(lv_touch_cont, touch_event_cb, LV_EVENT_RELEASED, NULL);

    /* Create Toolbar */
    lv_load_draw_dialog(lv_touch_cont);
}
```

2. `lv_load_draw_dialog()`: Creates the toolbar interface containing brush size selection, color selection, canvas reset, and toolbar visibility toggle.

(1) Create RST Reset Button: Triggers `lv_rst_event` callback upon click to clear all drawn content.

```

/* RST Button */
RST_btn = lv_btn_create(parent);
lv_obj_move_foreground(RST_btn);
lv_obj_set_size(RST_btn, 50, 30);
lv_obj_align(RST_btn, LV_ALIGN_TOP_RIGHT, -5, 5);
lv_obj_set_style_bg_color(RST_btn, lv_color_hex(0x666666), 0);
lv_obj_set_style_bg_opa(RST_btn, LV_OPA_COVER, 0);
lv_obj_t *RST_btn_label = lv_label_create(RST_btn);
lv_label_set_text(RST_btn_label, "RST");
lv_obj_center(RST_btn_label);
lv_obj_add_event_cb(RST_btn, lv_rst_event, LV_EVENT_CLICKED, NULL);

```

(2) Create Pen Toggle Button: Triggers `toggle_toolbar_event` callback upon click to control the show/hide state of the toolbar.

```

/* Toggle Toolbar Button (Pen Button) */
toggle_btn = lv_btn_create(parent);
lv_obj_move_foreground(toggle_btn);
lv_obj_set_size(toggle_btn, 50, 30);
lv_obj_align_to(toggle_btn, RST_btn, LV_ALIGN_OUT_BOTTOM_MID, 0, 10);
lv_obj_set_style_bg_color(toggle_btn, lv_color_hex(0x666666), 0);
lv_obj_set_style_bg_opa(toggle_btn, LV_OPA_COVER, 0);
lv_obj_t *toggle_btn_label = lv_label_create(toggle_btn);
lv_label_set_text(toggle_btn_label, "Pen");
lv_obj_center(toggle_btn_label);
lv_obj_add_event_cb(toggle_btn, toggle_toolbar_event, LV_EVENT_CLICKED, NULL);

```

(3) Create Toolbar Container: Sets a light gray background, black border, enables vertical scrolling, and holds the brush style selection widgets.

```

/* Create Toolbar Container */
toolbar_cont = lv_obj_create(parent);
lv_obj_set_size(toolbar_cont, toggle_btn_w, 140); /* width same as button, height 140 */
/*
/* Initial position directly below the Pen button */
lv_coord_t toolbar_y = toggle_btn_y + toggle_btn_h + 5; /* 5 pixels below button */
lv_obj_set_pos(toolbar_cont, toggle_btn_x, toolbar_y);

lv_obj_set_style_bg_color(toolbar_cont, lv_color_hex(0xE0E0E0), 0); /* Light gray
background */
lv_obj_set_style_bg_opa(toolbar_cont, LV_OPA_COVER, 0);
lv_obj_set_style_border_color(toolbar_cont, lv_color_hex(0x000000), 0); /* Black border */
/*
lv_obj_set_style_border_width(toolbar_cont, 2, 0);
lv_obj_set_style_pad_all(toolbar_cont, 5, 0); /* Padding */
/* Enable scrolling (Vertical) */
lv_obj_add_flag(toolbar_cont, LV_OBJ_FLAG_SCROLLABLE);
lv_obj_set_scroll_dir(toolbar_cont, LV_DIR_VER); /* Allow only vertical scrolling */

```

```
lv_obj_set_scrollbar_mode(toolbar_cont, LV_SCROLLBAR_MODE_AUTO); /* Auto show scrollbar */
lv_obj_move_foreground(toolbar_cont); /* Ensure it is on top */
toolbar_visible = 1; /* visible by default */
```

(4) Create Brush Size Selection Widgets: Uses 4 LED buttons to switch between 2/5/10/15 pixel brush sizes, binding corresponding callbacks.

```
/* Brush Size Label */
lv_obj_t *lv_pen_size = lv_label_create(toolbar_cont);
lv_label_set_text(lv_pen_size, "size");
lv_obj_align(lv_pen_size, LV_ALIGN_TOP_MID, 0, 5);

/* Brush Size Button 2 */
lv_pen_size_2 = lv_led_create(toolbar_cont);
lv_led_set_color(lv_pen_size_2, lv_color_hex(0xff0000));
lv_led_on(lv_pen_size_2);
lv_obj_align_to(lv_pen_size_2, lv_pen_size, LV_ALIGN_OUT_BOTTOM_MID, 0, 10);

lv_obj_t *lv_pen_size_2_label = lv_label_create(toolbar_cont);
lv_label_set_text(lv_pen_size_2_label, "2");
lv_obj_align_to(lv_pen_size_2_label, lv_pen_size_2, LV_ALIGN_OUT_LEFT_MID, -15, 0);

/* Brush Size Button 5 */
lv_pen_size_5 = lv_led_create(toolbar_cont);
lv_led_set_color(lv_pen_size_5, lv_color_hex(0xff0000));
lv_led_off(lv_pen_size_5);
lv_obj_align_to(lv_pen_size_5, lv_pen_size_2, LV_ALIGN_OUT_BOTTOM_MID, 0, 10);

lv_obj_t *lv_pen_size_5_label = lv_label_create(toolbar_cont);
lv_label_set_text(lv_pen_size_5_label, "5");
lv_obj_align_to(lv_pen_size_5_label, lv_pen_size_5, LV_ALIGN_OUT_LEFT_MID, -15, 0);

/* Brush Size Button 10 */
lv_pen_size_10 = lv_led_create(toolbar_cont);
lv_led_set_color(lv_pen_size_10, lv_color_hex(0xff0000));
lv_led_off(lv_pen_size_10);
lv_obj_align_to(lv_pen_size_10, lv_pen_size_5, LV_ALIGN_OUT_BOTTOM_MID, 0, 10);

lv_obj_t *lv_pen_size_10_label = lv_label_create(toolbar_cont);
lv_label_set_text(lv_pen_size_10_label, "10");
lv_obj_align_to(lv_pen_size_10_label, lv_pen_size_10, LV_ALIGN_OUT_LEFT_MID, -15, 0);

/* Brush Size Button 15 */
lv_pen_size_15 = lv_led_create(toolbar_cont);
lv_led_set_color(lv_pen_size_15, lv_color_hex(0xff0000));
lv_led_off(lv_pen_size_15);
lv_obj_align_to(lv_pen_size_15, lv_pen_size_10, LV_ALIGN_OUT_BOTTOM_MID, 0, 10);

lv_obj_t *lv_pen_size_15_label = lv_label_create(toolbar_cont);
lv_label_set_text(lv_pen_size_15_label, "15");
```

```
lv_obj_align_to(lv_pen_size_15_label, lv_pen_size_15, LV_ALIGN_OUT_LEFT_MID, -15, 0);

/* Set Brush Size Buttons as Clickable */
lv_obj_add_flag(lv_pen_size_2, LV_OBJ_FLAG_CLICKABLE);
lv_obj_add_flag(lv_pen_size_5, LV_OBJ_FLAG_CLICKABLE);
lv_obj_add_flag(lv_pen_size_10, LV_OBJ_FLAG_CLICKABLE);
lv_obj_add_flag(lv_pen_size_15, LV_OBJ_FLAG_CLICKABLE);

lv_obj_add_event_cb(lv_pen_size_2, lv_pen_size_2_event, LV_EVENT_CLICKED, NULL);
lv_obj_add_event_cb(lv_pen_size_5, lv_pen_size_5_event, LV_EVENT_CLICKED, NULL);
lv_obj_add_event_cb(lv_pen_size_10, lv_pen_size_10_event, LV_EVENT_CLICKED, NULL);
lv_obj_add_event_cb(lv_pen_size_15, lv_pen_size_15_event, LV_EVENT_CLICKED, NULL);
```

(5) Create Brush Color Selection Widgets: Uses 4 LED buttons to switch between Red/Green/Blue/Yellow, binding corresponding callbacks.

```
/* Color Label */
lv_obj_t *lv_pen_color = lv_label_create(toolbar_cont);
lv_label_set_text(lv_pen_color, "color");
lv_obj_align_to(lv_pen_color, lv_pen_size, LV_ALIGN_OUT_BOTTOM_MID, 0, 150);

/* Red Button */
lv_pen_color_red = lv_led_create(toolbar_cont);
lv_led_set_color(lv_pen_color_red, lv_color_hex(0xff0000));
lv_led_on(lv_pen_color_red);
lv_obj_set_style_shadow_width(lv_pen_color_red, 0, LV_STATE_DEFAULT);
lv_obj_align_to(lv_pen_color_red, lv_pen_color, LV_ALIGN_OUT_BOTTOM_MID, 0, 10);

/* Green Button */
lv_pen_color_green = lv_led_create(toolbar_cont);
lv_led_set_color(lv_pen_color_green, lv_color_hex(0x00ff00));
lv_led_off(lv_pen_color_green);
lv_obj_align_to(lv_pen_color_green, lv_pen_color_red, LV_ALIGN_OUT_BOTTOM_MID, 0, 10);

/* Blue Button */
lv_pen_color_blue = lv_led_create(toolbar_cont);
lv_led_set_color(lv_pen_color_blue, lv_color_hex(0x0000ff));
lv_led_off(lv_pen_color_blue);
lv_obj_align_to(lv_pen_color_blue, lv_pen_color_green, LV_ALIGN_OUT_BOTTOM_MID, 0, 10);

/* Yellow Button */
lv_pen_color_yellow = lv_led_create(toolbar_cont);
lv_led_set_color(lv_pen_color_yellow, lv_color_hex(0xffff00));
lv_led_off(lv_pen_color_yellow);
lv_obj_align_to(lv_pen_color_yellow, lv_pen_color_blue, LV_ALIGN_OUT_BOTTOM_MID, 0, 10);

/* Set Color Buttons as Clickable */
lv_obj_add_flag(lv_pen_color_red, LV_OBJ_FLAG_CLICKABLE);
lv_obj_add_flag(lv_pen_color_green, LV_OBJ_FLAG_CLICKABLE);
lv_obj_add_flag(lv_pen_color_blue, LV_OBJ_FLAG_CLICKABLE);
lv_obj_add_flag(lv_pen_color_yellow, LV_OBJ_FLAG_CLICKABLE);
```

(6) Set clickable flags for all interactive buttons and bind corresponding event callback functions.

```
lv_obj_add_event_cb(lv_pen_color_red, lv_pen_color_red_event, LV_EVENT_CLICKED, NULL);
lv_obj_add_event_cb(lv_pen_color_green, lv_pen_color_green_event, LV_EVENT_CLICKED,
NULL);
lv_obj_add_event_cb(lv_pen_color_blue, lv_pen_color_blue_event, LV_EVENT_CLICKED, NULL);
lv_obj_add_event_cb(lv_pen_color_yellow, lv_pen_color_yellow_event, LV_EVENT_CLICKED,
NULL);
```

3. `touch_event_cb()`: Touch event handling callback function.

(1) `if (code != LV_EVENT_PRESSED && code != LV_EVENT_PRESSING && code != LV_EVENT_RELEASED)`: Processes only Pressed, Pressing, and Released events. Drawing executes only when the touch target is the canvas container.

(2) Calls `start_new_stroke` to create a new drawing trajectory, which is the LVGL line object.

(3) Calls `add_point_to_stroke` to add new coordinate points to the current trajectory, achieving continuous line drawing.

(4) Calls `finish_stroke` to manage the storage of the current drawing trajectory.

```
static void touch_event_cb(lv_event_t *e)
{
    lv_event_code_t code = lv_event_get_code(e);

    /* Process only touch-related events */
    if (code != LV_EVENT_PRESSED && code != LV_EVENT_PRESSING && code != LV_EVENT_RELEASED)
    {
        return;
    }

    /* Key: Process drawing only if the event target is the lv_touch_cont container itself */
    lv_obj_t *target = lv_event_get_target(e);
    if (target != lv_touch_cont)
    {
        return;
    }

    // Code omitted: Get touch coordinates, check canvas area
    if (screen_x >= 0 && screen_x < canvas_w && screen_y >= 0 && screen_y < scr_h)
    {
        if (code == LV_EVENT_PRESSED)
        {
            /* Pressed: Start a new stroke */
            start_new_stroke(lv_touch_cont, screen_x, screen_y, pen_size,
POINT_COLOR_TBL[pen_color_idx]);
            last_x = screen_x;
            last_y = screen_y;
        }
    }
}
```

```

else if (code == LV_EVENT_PRESSING)
{
    /* Dragging: Add new point to current stroke */
    // Distance threshold logic omitted
    add_point_to_stroke(screen_x, screen_y);
    last_x = screen_x;
    last_y = screen_y;
}
else if (code == LV_EVENT_RELEASED)
{
    /* Released: Finish current stroke */
    finish_stroke();
    last_x = -1;
    last_y = -1;
}
}
}

```

4. `start_new_stroke()` : Function to create a new drawing trajectory.

- (1) Calls `cleanup_old_lines` to clear old trajectories if the limit is exceeded, avoiding memory overflow.
- (2) `current_line = lv_line_create(parent)` : Creates a new line object as the drawing carrier and allocates an independent style object for it.
- (3) `current_style = lv_mem_alloc(sizeof(lv_style_t))` : Configures the line object's style based on the currently selected brush settings, such as width, color, opacity, and rounded ends.
- (4) `current_points = lv_mem_alloc(sizeof(lv_point_t) * current_point_capacity)` : Allocates memory for the coordinate point array, initializes the first touch point coordinate, and binds it to the line object.

```

static void start_new_stroke(lv_obj_t *parent, lv_coord_t x, lv_coord_t y, uint8_t width,
uint16_t color_rgb565)
{
    /* Clean up old objects */
    cleanup_old_lines();

    /* Create new line object */
    current_line = lv_line_create(parent);

    /* Create independent style for each line object (Avoid style sharing causing color
pollution) */
    current_style = lv_mem_alloc(sizeof(lv_style_t));
    lv_style_init(current_style);
    lv_style_set_line_width(current_style, width);
    // Color conversion logic omitted
    lv_style_set_line_color(current_style, color);
    lv_style_set_line_opa(current_style, LV_OPA_COVER);
    lv_style_set_line_rounded(current_style, true);
    lv_obj_add_style(current_line, current_style, 0);

    /* Allocate independent points array for the current stroke */
    current_point_capacity = 100;
    current_points = lv_mem_alloc(sizeof(lv_point_t) * current_point_capacity);
}

```

```

    current_point_count = 1;
    current_points[0].x = x;
    current_points[0].y = y;
    lv_line_set_points(current_line, current_points, current_point_count);
}

```

5. `add_point_to_stroke()`: Adds new coordinate points to the current drawing trajectory's point array during touch dragging, achieving continuous line drawing.

```

static void add_point_to_stroke(lv_coord_t x, lv_coord_t y)
{
    if (current_line == NULL || current_points == NULL)
    {
        return;
    }

    /* If capacity is insufficient, expand */
    if (current_point_count >= current_point_capacity)
    {
        current_point_capacity *= 2;
        current_points = lv_mem_realloc(current_points, sizeof(lv_point_t) *
current_point_capacity);
    }

    /* Add new point */
    current_points[current_point_count].x = x;
    current_points[current_point_count].y = y;
    current_point_count++;

    /* Update points of the line object */
    lv_line_set_points(current_line, current_points, current_point_count);
}

```

6. `finish_stroke()`: Saves the current line object, point array, and style object into the `line_objects_data` management array, and resets current temporary drawing variables to prepare for the next stroke.

```

static void finish_stroke(void)
{
    if (current_line != NULL && line_object_count < MAX_LINE_OBJECTS && current_points !=
NULL)
    {
        /* Save line object, points array, and style pointer */
        line_objects_data[line_object_count].line_obj = current_line;
        line_objects_data[line_object_count].points = current_points;
        line_objects_data[line_object_count].point_count = current_point_count;
        line_objects_data[line_object_count].style = current_style;
        current_style = NULL; /* Style saved, clear current style pointer */
    }
}

```

```

        line_object_count++;
        current_line = NULL;
        current_points = NULL;
        current_point_count = 0;
        current_point_capacity = 0;
    }
}

```

7. `cleanup_old_lines()`: Automatically cleans up the oldest half of the trajectories when the number of drawing trajectories reaches the limit, releasing corresponding memory to avoid leaks.

```

static void cleanup_old_lines(void)
{
    if (line_object_count >= MAX_LINE_OBJECTS)
    {
        uint16_t delete_count = MAX_LINE_OBJECTS / 2;
        for (uint16_t i = 0; i < delete_count; i++)
        {
            if (line_objects_data[i].line_obj != NULL)
            {
                delete_line_with_style(line_objects_data[i].line_obj);
                /* Free points array and style */
                if (line_objects_data[i].points != NULL)
                {
                    lv_mem_free(line_objects_data[i].points);
                }
                if (line_objects_data[i].style != NULL)
                {
                    lv_style_reset(line_objects_data[i].style);
                    lv_mem_free(line_objects_data[i].style);
                }
                // Reset management array element
            }
        }
        /* Move remaining objects forward */
        for (uint16_t i = 0; i < line_object_count - delete_count; i++)
        {
            line_objects_data[i] = line_objects_data[i + delete_count];
        }
        line_object_count -= delete_count;
    }
}

```

8. `delete_line_with_style()`: Used to safely delete a line object by first removing all styles and then deleting the object, preventing LVGL style management memory leaks.

```
static void delete_line_with_style(lv_obj_t *line_obj)
{
    if (line_obj == NULL) return;
    /* Remove all styles first so LVGL cleans up correctly */
    lv_obj_remove_style_all(line_obj);
    /* Then delete the object */
    lv_obj_del(line_obj);
}
```

9. `lv_rst_event()`: Triggered when the RST button is clicked. It clears all drawing trajectories, frees corresponding memory, and resets drawing status.

```
static void lv_rst_event(lv_event_t *e)
{
    if (lv_event_get_code(e) == LV_EVENT_CLICKED)
    {
        // Code omitted: Release current temporary drawing resources
        /* Clear all completed line objects */
        for (uint16_t i = 0; i < line_object_count; i++)
        {
            // Omitted: Free style, delete line object, free points array
        }
        // Omitted: Reset status variables, delete residual line objects in canvas, refresh
display
    }
}
```

10. `toggle_toolbar_event()`: Triggered when the Pen button is clicked. It toggles the toolbar's display/hide status by adjusting the toolbar coordinates.

```
static void toggle_toolbar_event(lv_event_t *e)
{
    if (lv_event_get_code(e) != LV_EVENT_CLICKED)
    {
        return;
    }

    toolbar_visible = !toolbar_visible;

    if (toolbar_cont != NULL && toggle_btn != NULL)
    {
        // Omitted: Get button coordinates and size
        if (toolbar_visible)
        {
            /* Show toolbar: Move to directly below the Pen button */
            lv_coord_t toolbar_y = toggle_btn_y + toggle_btn_h + 5;
            lv_obj_set_pos(toolbar_cont, toggle_btn_x, toolbar_y);
            lv_obj_clear_flag(toolbar_cont, LV_OBJ_FLAG_HIDDEN);
        }
    }
}
```

```

        else
        {
            /* Hide toolbar: Move outside the screen bottom */
            lv_obj_set_pos(toolbar_cont, toggle_btn_x, scr_h);
        }
    }
}

```

11. Brush Size Toggle Callback: Includes `lv_pen_size_2_event`, `lv_pen_size_5_event`, `lv_pen_size_10_event`, and `lv_pen_size_15_event`.

Clicking the corresponding size button sets the `pen_size` global variable and indicates the currently selected size via LED status.

Example `lv_pen_size_2_event`:

```

static void lv_pen_size_2_event(lv_event_t *e)
{
    if (lv_event_get_code(e) == LV_EVENT_CLICKED)
    {
        pen_size = 2;
        lv_led_on(lv_pen_size_2);
        lv_led_off(lv_pen_size_5);
        lv_led_off(lv_pen_size_10);
        lv_led_off(lv_pen_size_15);
    }
}

```

12. Brush Color Toggle Callback: Includes `lv_pen_color_red_event`, `lv_pen_color_green_event`, `lv_pen_color_blue_event`, and `lv_pen_color_yellow_event`. Clicking the corresponding color button sets the `pen_color_idx` global variable and indicates the currently selected color via LED status.

Example `lv_pen_color_red_event`:

```

static void lv_pen_color_red_event(lv_event_t *e)
{
    if (lv_event_get_code(e) == LV_EVENT_CLICKED)
    {
        pen_color_idx = 0;
        lv_led_on(lv_pen_color_red);
        lv_led_off(lv_pen_color_green);
        lv_led_off(lv_pen_color_blue);
        lv_led_off(lv_pen_color_yellow);
    }
}

```

5.13 LVGL File System Depth Application Experiment

5.13.1 Experiment Introduction

This experiment implements a simple file management system based on the LVGL graphics library and the FATFS file system. It generates a file list by scanning a specified directory on the storage device, distinguishing between folders, image files, and standard files by matching corresponding icons. It allows entering folders, reading and displaying TXT file content, and loading BIN images by binding click events to list items. Additionally, Menu/Return buttons are designed for path navigation and interface resets. Combined with LVGL pages, list widgets, and file IO operations, this experiment validates the visualization logic of file management and the parsing and display methods for different file types in embedded systems.

5.13.2 Program Download

Locate the [Source Code](#) -> **13_lvgl_file_system** directory under the same path as this document and click **Select Folder**.

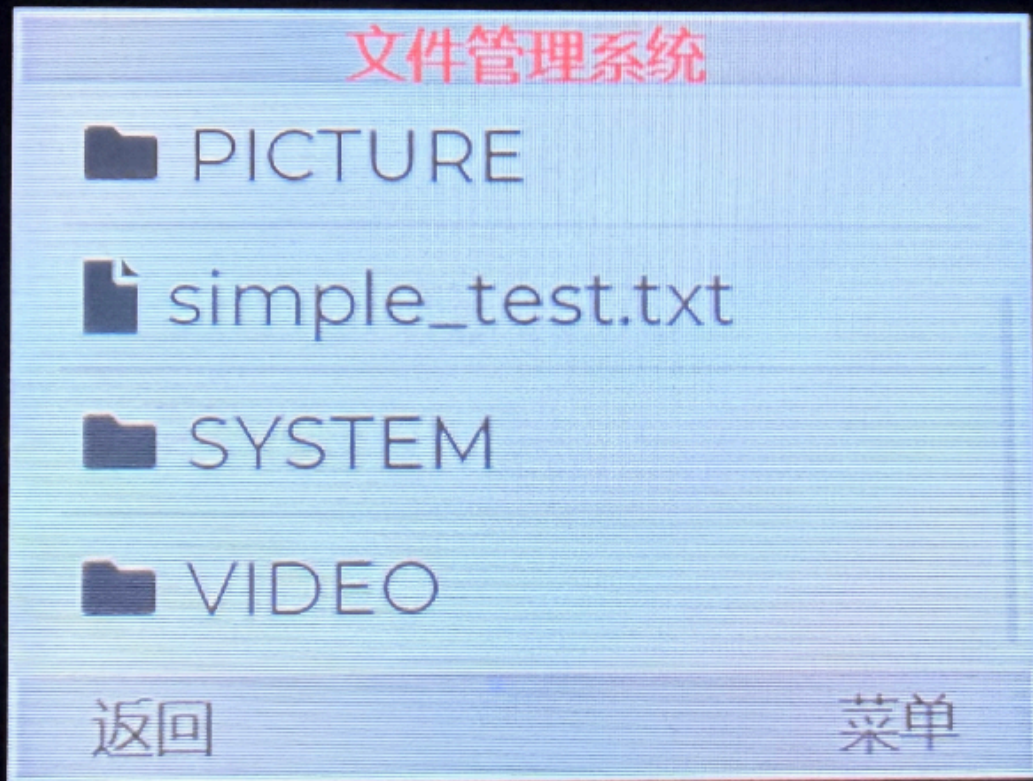
Note

Ensure the program file path contains only English characters, as non-English characters may cause compilation failures.

For specific Program Download Instructions (Must Read), refer to [5.1 Program Download Instructions \(Must Read\)](#).

5.13.3 Program Outcome

Upon running the program, the top of the screen displays the "File Management System" title bar, the bottom features "Return" and "Menu" function keys, and the middle area shows the file list of the storage device's root directory. Clicking a folder enters the corresponding directory and updates the list content in real-time. Clicking a TXT file opens a new page displaying the text content. Clicking "Return" inside the page closes the current view and traces back the file path. Clicking the "Menu" button at the bottom resets the list and returns to the root directory, while the "Return" button goes back to the previous directory. Throughout the operation, file paths are managed via flags, and list item interactions and page transitions are smooth with accurate file type icon matching. The specific effect is shown below:



5.13.4 Program Analysis

Note

`lv_mainstart.c` is the file in the LVGL application project that most intuitively reflects the experiment's effects. Therefore, this chapter focuses on analyzing the program within

`lv_mainstart.c` to help directly understand the core functions of the entire project.

- **lv_mainstart.c**

1. `lv_tell` is a helper function to retrieve the current position of the file pointer. It uses the `lv_fs_tell` interface to get the file's current offset, prints it, and returns the value.

```
long lv_tell(lv_fs_file_t *fd)
{
    uint32_t pos = 0;
    lv_fs_tell(fd, &pos);
    printf("\nlv_tcur pos is: %ld\n", pos);
    return pos;
}
```

2. `lv_file_read` is the core function for reading file content. It reads the content of a file at a specified path. It first attempts to open the file in read-only mode, returning an error if it fails. Upon success, it retrieves and adjusts the file pointer, reads the file content into a preset buffer, closes the file upon completion, and returns the result.

```

lv_fs_res_t lv_file_read(const char *path)
{
    uint32_t rsize = 0;
    lv_fs_file_t fd;
    lv_fs_res_t res;

    res = lv_fs_open(&fd, path, LV_FS_MODE_RD);

    if (res != LV_FS_RES_OK)
    {
        printf("open %s ERROR\n", path);
        return LV_FS_RES_UNKNOWN;
    }

    lv_tell(&fd);
    lv_fs_seek(&fd, 0, LV_FS_SEEK_SET);
    lv_tell(&fd);
    res = lv_fs_read(&fd, lv_flie.rbuf, FILE_SEZE, &rsize);

    if (res != LV_FS_RES_OK)
    {
        printf("read %s ERROR\n", path);
        return LV_FS_RES_UNKNOWN;
    }

    lv_tell(&fd);

    lv_fs_close(&fd);

    return LV_FS_RES_OK;
}

```

3. `lv_btn_close_event` is the event callback for the page return button. It responds to interaction events. When a button release event is detected, if an image object exists, it deletes it and sets the pointer to null. It then adjusts the file return flag and finally deletes the current page container.

```

void lv_btn_close_event(lv_event_t *event)
{
    lv_event_code_t code = lv_event_get_code(event);
    lv_obj_t * obj = lv_event_get_target(event);

    if (code == LV_EVENT_RELEASED)
    {
        if (lv_flie.lv_image_read != NULL) /* Check if image path is null */
        {
            lv_obj_del(lv_flie.lv_image_read); /* Delete image object */
            lv_flie.lv_image_read = NULL; /* Set image object to NULL */
        }

        lv_flie.lv_prev_file_flag -- ; /* Decrement file return flag */
    }
}

```

```

        lv_obj_del(lv_flie.lv_page_cont);    /* Delete page when return is pressed */
    }
}

```

4. `lv_create_page` creates a page for displaying file content. It creates a page container within the specified parent object, setting its size, style, and alignment. It then creates a display area for the return button and a clickable "Return" label, binds the return event callback to the label, and finally returns the created page container.

```

lv_obj_t *lv_create_page(lv_obj_t *parent)
{
    lv_flie.lv_page_cont = lv_obj_create(parent);    /* Create
container */
    lv_obj_set_size(lv_flie.lv_page_cont, lcddev.width, lcddev.height);
    lv_obj_set_style_radius(lv_flie.lv_page_cont, 0, LV_STATE_DEFAULT);    /* Set radius
to 0 */
    lv_obj_clear_flag(lv_flie.lv_page_cont, LV_OBJ_FLAG_SCROLL_CHAIN_HOR);
    lv_obj_clear_flag(lv_flie.lv_page_cont, LV_OBJ_FLAG_SCROLL_CHAIN_VER);
    lv_obj_align_to(lv_flie.lv_page_cont, parent, LV_ALIGN_CENTER, 0, 0);

    lv_obj_t *lv_page_obj = lv_obj_create(lv_flie.lv_page_cont);    /* Create return button
area */
    lv_obj_set_style_bg_color(lv_page_obj, lv_palette_main(LV_PALETTE_BLUE),
LV_STATE_DEFAULT);
    lv_obj_align(lv_page_obj, LV_ALIGN_BOTTOM_MID, 0, 10);
    lv_obj_set_size(lv_page_obj, lcddev.width, myFont24.line_height);

    lv_obj_t *lv_page_back_btn = lv_label_create(lv_page_obj);    /* Create label as
return object */
    lv_obj_set_style_text_font(lv_page_back_btn, &myFont24, LV_STATE_DEFAULT);
    lv_label_set_text(lv_page_back_btn, "Return");
    lv_obj_align_to(lv_page_back_btn, NULL, LV_ALIGN_CENTER, 0, 0);
    lv_obj_add_flag(lv_page_back_btn, LV_OBJ_FLAG_CLICKABLE);    /* Make label clickable
*/
    lv_obj_add_event_cb(lv_page_back_btn, lv_btn_close_event, LV_EVENT_ALL, NULL);    /* Set
callback */

    return lv_flie.lv_page_cont;
}

```

5. `lv_show_filetext` displays the content of text files. It creates a label widget within the specified parent object, configures the label's text wrapping mode, font, and width, sets the label text to the content of the read buffer, and finally clears the buffer.

```

void lv_show_filetxt(lv_obj_t *parent)
{
    lv_obj_t *label = lv_label_create(parent);
    lv_label_set_long_mode(label, LV_LABEL_LONG_WRAP);
    lv_obj_set_style_text_font(label, &lv_font_montserrat_18, LV_STATE_DEFAULT);
    lv_obj_set_width(label, lv_obj_get_width(parent));
    lv_label_set_text(label, (char *)lv_flie.rbuf); /* Display read data */
    memset(lv_flie.rbuf, 0, sizeof(lv_flie.rbuf));
}

```

6. `lv_show_imgbin` displays BIN format images. It creates an image widget within the specified parent object, sets the image source to the specified file path, and aligns the widget to the center of the parent object.

```

void lv_show_imgbin(lv_obj_t *parent, const char *path)
{
    lv_flie.lv_image_read = lv_img_create(parent); /* Create image
widget */
    lv_img_set_src(lv_flie.lv_image_read, path); /* Set image for
widget */
    // lv_img_set_auto_size(lv_flie.lv_image_read, true); /* Auto-size */
    lv_obj_align_to(lv_flie.lv_image_read, parent, LV_ALIGN_CENTER, 0, 0); /* Align widget
relative to parent */
}

```

7. `lv_pash_joint` handles file path concatenation. It first saves the current path to the history array and increments the path flag. Then, using string copy and concatenation, it combines the directory path, separator, and filename into a complete file path, which it returns.

```

char *lv_pash_joint(void)
{
    lv_flie.lv_prev_file[lv_flie.lv_prev_file_flag] = (char *)lv_flie.lv_pash; /* Save
previous path to array */
    lv_flie.lv_prev_file_flag++; /* Increment previous path
count flag */

    strcpy((char *)lv_flie.pname, lv_flie.lv_pash); /* Copy path (directory) */
    strcat((char *)lv_flie.pname, "/"); /* Copy path (directory) */
    strcat((char *)lv_flie.pname, (char *)lv_flie.lv_pname); /* Append filename */
    return lv_flie.pname;
}

```

8. `lv_list_btn_event` is the event callback for file list items. It responds to item clicks by polling to identify the clicked object. It allocates memory for the filename and full path, then checks the file extension to determine if it is a folder or a file. If it is a folder, it concatenates the path, deletes the old list, and rescans. If it is a TXT or BIN file, it creates a display page and loads the content. Finally, it restores the file suffix flag.

```

static void lv_list_btn_event(lv_event_t *event)
{
    lv_event_code_t code = lv_event_get_code(event);
    lv_obj_t *obj = lv_event_get_target(event);

    if(code == LV_EVENT_CLICKED)
    {
        for (int i = 0; i <= lv_flie.list_flie_nuber ;i++) /* Poll list items */
        {
            if (obj == lv_flie.list_btn[i]) /* Identify which button was pressed */
            {
                lv_flie.lv_pname = malloc(255);          /* Allocate memory for filename */
                lv_flie.pname = malloc(255);              /* Allocate memory for full path
name */

                lv_flie.lv_pname = (char *)lv_list_get_btn_text(lv_flie.list,
lv_flie.list_btn[i]); /* Get list item value */
                for (int suffix = 0; suffix < LV_SUFFIX(lv_suffix); suffix++) /* Check
file suffixes */
                {
                    if (strstr(lv_flie.lv_pname, lv_suffix[suffix]) != NULL) /* If
not a folder */
                    {
                        lv_flie.lv_suffix_flag = 0; /* Set
suffix flag to 0 */
                        break;
                    }
                }

                if (lv_flie.lv_suffix_flag == 1) /* Flag
not 0 implies folder operation */
                {
                    lv_flie.lv_pash = lv_pash_join(); /* Pass
file path to lv_pash */
                    lv_del_list(lv_flie.list); /* Delete
list */
                    lv_scan_files(lv_flie.pname, lv_scr_act()); /* Re-
create file list */
                }
                else
                {
                    if (strstr(lv_flie.lv_pname, ".txt") != NULL) /* If not
folder and is txt file */
                    {
                        if (lv_file_read(lv_pash_join()) == LV_FS_RES_OK) /* Check
if read txt successful */
                        {
                            lv_flie.lv_return_page = lv_create_page(lv_scr_act()); /* Create
page */
                            lv_show_filetxt(lv_flie.lv_return_page); /* Display
txt data on page */
                        }
                    }
                }
            }
        }
    }
}

```



```

        ||strstr(lv_flie.SD_fno.fname, ".gif") != NULL
        ||strstr(lv_flie.SD_fno.fname, ".avi") != NULL)
    {
        lv_flie.image_scr = LV_SYMBOL_IMAGE;
    }
    else
    {
        lv_flie.image_scr = LV_SYMBOL_FILE;
    }

    lv_flie.list_btn[lv_flie.list_flie_nuber] = lv_list_add_btn(lv_flie.list,
lv_flie.image_scr, lv_flie.SD_fno.fname);    /* Add list item (File) */
    }

    lv_obj_set_style_pad_left(lv_flie.list_btn[lv_flie.list_flie_nuber], 5,
LV_STATE_DEFAULT);    /* Set list item left padding */
    lv_obj_set_style_pad_right(lv_flie.list_btn[lv_flie.list_flie_nuber], 5,
LV_STATE_DEFAULT);    /* Set list item right padding */
    lv_obj_add_event_cb(lv_flie.list_btn[lv_flie.list_flie_nuber],
lv_list_btn_event, LV_EVENT_ALL, NULL); /* Set list callback */
    }

    f_closedir(&lv_flie.lv_dir); /* Close file directory */
    }

    return lv_flie.fr;    /* Return */
}

```

10. `lv_animation` is the function responsible for configuring the entrance animation for the list widget. It defines and initializes the animation object, specifies the target widget and execution function, sets the start position to the screen width and the end position to 0, and configures the duration to 300 milliseconds before starting the animation, thereby adding a sliding entrance visual effect to the list.

```

void lv_animation(lv_obj_t *parent)
{
    lv_anim_t a;                                /* Step 1: Define an
animation */
    lv_anim_init(&a);                            /* Step 2: Initialize
animation */
    lv_anim_set_var(&a, parent);                 /* Step 3: Set target widget
*/
    lv_anim_set_exec_cb(&a, (lv_anim_exec_xcb_t)lv_obj_set_x); /* Set animation function
var */
    int32_t start = lv_obj_get_width(lv_scr_act());
    int32_t end = 0;
    lv_anim_set_values(&a, start, end);          /* Set start and end values
*/
    lv_anim_set_time(&a, 300);                  /* Set duration 300ms */
    lv_anim_start(&a);                          /* Step 4: Start animation
*/
}

```

11. `lv_create_list` is the function responsible for creating the file list widget. It initializes the list widget within the specified parent object, configures the list's dimensions, alignment mode, display font, and corner radius style, and aligns the list directly below the return button area.

```

void lv_create_list(lv_obj_t *parent)
{
    lv_flie.list = lv_list_create(parent); /* Create list */
    // lv_animation(lv_flie.list);
    lv_obj_set_size(lv_flie.list, lcddev.width, lcddev.height - myFont24.line_height * 2 -
10); /* Set list size */
    lv_obj_align_to(lv_flie.list, lv_flie.lv_page_obj, LV_ALIGN_OUT_BOTTOM_LEFT, 0, 0);
    /* Set list alignment */
    lv_obj_set_style_text_font(lv_flie.list, &lv_font_montserrat_24, LV_STATE_DEFAULT);
    /* Set font */
    lv_obj_set_style_radius(lv_flie.list, 0, LV_STATE_DEFAULT);
    /* Set radius to 0 */
}

```

12. `lv_page_tile` is the function responsible for creating the page title bar. It initializes a title bar container within the specified parent object, configures the container's dimensions, background color, and corner radius style, and aligns it to the top of the screen. Furthermore, it creates a label widget within the title bar, sets the label text to "File Management System" along with the font and text color, and aligns it to the center of the title bar.

```

void lv_page_tile(lv_obj_t *parent)
{
    lv_flie.lv_page_obj = lv_obj_create(parent);
    lv_obj_set_size(lv_flie.lv_page_obj, lcddev.width, myFont24.line_height);
    lv_obj_set_style_bg_color(lv_flie.lv_page_obj, lv_palette_main(LV_PALETTE_GREY),
LV_STATE_DEFAULT);
}

```

```

lv_obj_set_style_radius(lv_flie.lv_page_obj, 0, LV_STATE_DEFAULT);          /* Set
radius to 0 */
lv_obj_align_to(lv_flie.lv_page_obj, parent, LV_ALIGN_TOP_LEFT, 0, 0);

lv_obj_t *lv_page_label = lv_label_create(lv_flie.lv_page_obj);
lv_label_set_text(lv_page_label, "File Management System");
lv_obj_set_style_text_color(lv_page_label, lv_palette_main(LV_PALETTE_RED),
LV_STATE_DEFAULT);
lv_obj_set_style_text_font(lv_page_label, &myFont24, LV_STATE_DEFAULT);
lv_obj_align_to(lv_page_label, lv_flie.lv_page_obj, LV_ALIGN_CENTER, 0, 0);
}

```

13. `lv_back_btn_event_handler` serves as the event callback function for the "Menu" and "Return" buttons within the file system, responsible for detecting short-press events. If the "Menu" button is activated, the function deletes the current list and re-initializes the list. If the "Return" button is activated, it adjusts the historical path flag, deletes the current list, and scans the files located in the parent directory.

```

void lv_back_btn_event_handler(lv_event_t *event)
{
    lv_event_code_t code = lv_event_get_code(event);
    lv_obj_t *obj = lv_event_get_target(event);

    if(code == LV_EVENT_SHORT_CLICKED)
    {
        if (obj == lv_flie.lv_back_btn)
        {
            lv_del_list(lv_flie.list);          /* Delete list */
            list_init(lv_scr_act());           /* Return to Menu */
        }
        if (obj == lv_flie.lv_prev_btn)
        {
            lv_flie.lv_prev_file_flag--;      /* Decrement file return flag */

            if (lv_flie.lv_prev_file_flag <= 0)
            {
                lv_flie.lv_prev_file_flag = 0; /* Force to 0 if <= 0 */
            }

            lv_del_list(lv_flie.list);          /* Immediately delete list widget */
            lv_flie.lv_pash = lv_flie.lv_prev_file[lv_flie.lv_prev_file_flag];
            /* Copy parent path to lv_pash */
            lv_scan_files(lv_flie.lv_prev_file[lv_flie.lv_prev_file_flag], lv_scr_act());
            /* Read file path */
        }
    }
}

```

14. `lv_general_win_create` is the function responsible for creating general file system buttons. It initializes "Menu" and "Back" label widgets within the specified parent object, configures the display font, text content, and alignment for the labels, adds clickable flags to them, and binds the event callback functions.

```

void lv_general_win_create(lv_obj_t *parent)
{
    lv_flie.lv_back_btn = lv_label_create(parent);
    lv_obj_set_style_text_font(lv_flie.lv_back_btn, &myFont24, LV_STATE_DEFAULT);    /* Set
font */

    lv_label_set_text(lv_flie.lv_back_btn, "Menu");
    lv_obj_align_to(lv_flie.lv_back_btn, parent, LV_ALIGN_RIGHT_MID, -10, 0);
    lv_obj_add_flag(lv_flie.lv_back_btn, LV_OBJ_FLAG_CLICKABLE);                                /* Make
label clickable */
    lv_obj_add_event_cb(lv_flie.lv_back_btn, lv_back_btn_event_handler, LV_EVENT_ALL, NULL);
    /* Set callback */
    lv_flie.lv_prev_btn = lv_label_create(parent);
    lv_obj_set_style_text_font(lv_flie.lv_prev_btn, &myFont24, LV_STATE_DEFAULT);
    lv_label_set_text(lv_flie.lv_prev_btn, "Back");
    lv_obj_align_to(lv_flie.lv_prev_btn, parent, LV_ALIGN_LEFT_MID, 10, 0);
    lv_obj_add_flag(lv_flie.lv_prev_btn, LV_OBJ_FLAG_CLICKABLE);                                /* Make label
clickable */
    lv_obj_add_event_cb(lv_flie.lv_prev_btn, lv_back_btn_event_handler, LV_EVENT_ALL, NULL);
}

```

15. `lv_page_back` is the function responsible for creating the return button area of the file system. It initializes a return button container within the specified parent object, configures the container's dimensions, background color, and corner radius style, and aligns it to the bottom of the screen. It then invokes the general button creation function to generate the "Menu" and "Return" buttons within this container.

```

void lv_page_back(lv_obj_t *parent)
{
    lv_flie.lv_back_obj = lv_obj_create(parent);
    /* Create return object area */
    lv_obj_set_size(lv_flie.lv_back_obj, lcddev.width, myFont24.line_height + 10);
    /* Set area size */
    lv_obj_set_style_bg_color(lv_flie.lv_back_obj, lv_palette_main(LV_PALETTE_GREY),
LV_STATE_DEFAULT);    /* Set area color to grey */
    lv_obj_set_style_radius(lv_flie.lv_back_obj, 0, LV_STATE_DEFAULT);
    /* Set radius to 0 */
    lv_obj_align_to(lv_flie.lv_back_obj, parent, LV_ALIGN_BOTTOM_MID, 0, 0);
    /* Set alignment */
    lv_general_win_create(lv_flie.lv_back_obj);
    /* Create return and menu buttons */
}

```

16. `list_init` serves as the initialization function for the file list. It sets the initial path of the file management system to the SD card root directory "0:", resets global parameters such as the history path flag, file count statistics, and the file suffix flag, and invokes the file scanning function to load the list of files and folders within the initial path.

```

void list_init(lv_obj_t *parent)
{
    lv_flie.lv_pash = "0:"; /* Initial path */
    lv_flie.lv_prev_file_flag = 0; /* Clear previous path flag */
    lv_flie.lv_prev_file[lv_flie.lv_prev_file_flag] = "0:"; /* Initial previous folder path */
    lv_flie.list_flie_nuber = 0; /* Initial file count */
    lv_flie.lv_suffix_flag = 1; /* File suffix flag */
    lv_scan_files(lv_flie.lv_pash, parent); /* Read file path */
}

```

17. `lv_mainstart` serves as the entry point function for the file management system. It sequentially invokes the functions responsible for creating the page title, establishing the return button area, and initializing the file list, thereby completing the interface construction and initial data loading. It acts as the main entry point for launching the entire file management functionality.

```

void lv_mainstart(void)
{
    lv_page_tile(lv_scr_act()); /* Set page title */
    lv_page_back(lv_scr_act()); /* Set page return */
    list_init(lv_scr_act()); /* List initialization */
}

```

5.14 LVGL Data Format Display Conversion Experiment

5.14.1 Experiment Introduction

Based on the LVGL graphics library, this experiment implements the mutual conversion functionality among binary, decimal, hexadecimal, and octal formats. It achieves intuitive modification of binary bits by creating a 33-bit visual binary bit button group. Paired with customized virtual keyboards, it provides targeted input restrictions for the text input fields of the four numeral systems. By embedding base conversion algorithms within event callback functions, the system realizes real-time synchronous updates of values across all bases following binary bit clicks or numerical input in the text fields. This validates the visual interaction and algorithm implementation logic for base conversion within an embedded graphical interface.

5.14.2 Program Download

Locate the [Source Code](#) -> `14_lvgl_conversion_of_number` directory under the same path as this document and click **Select Folder**.

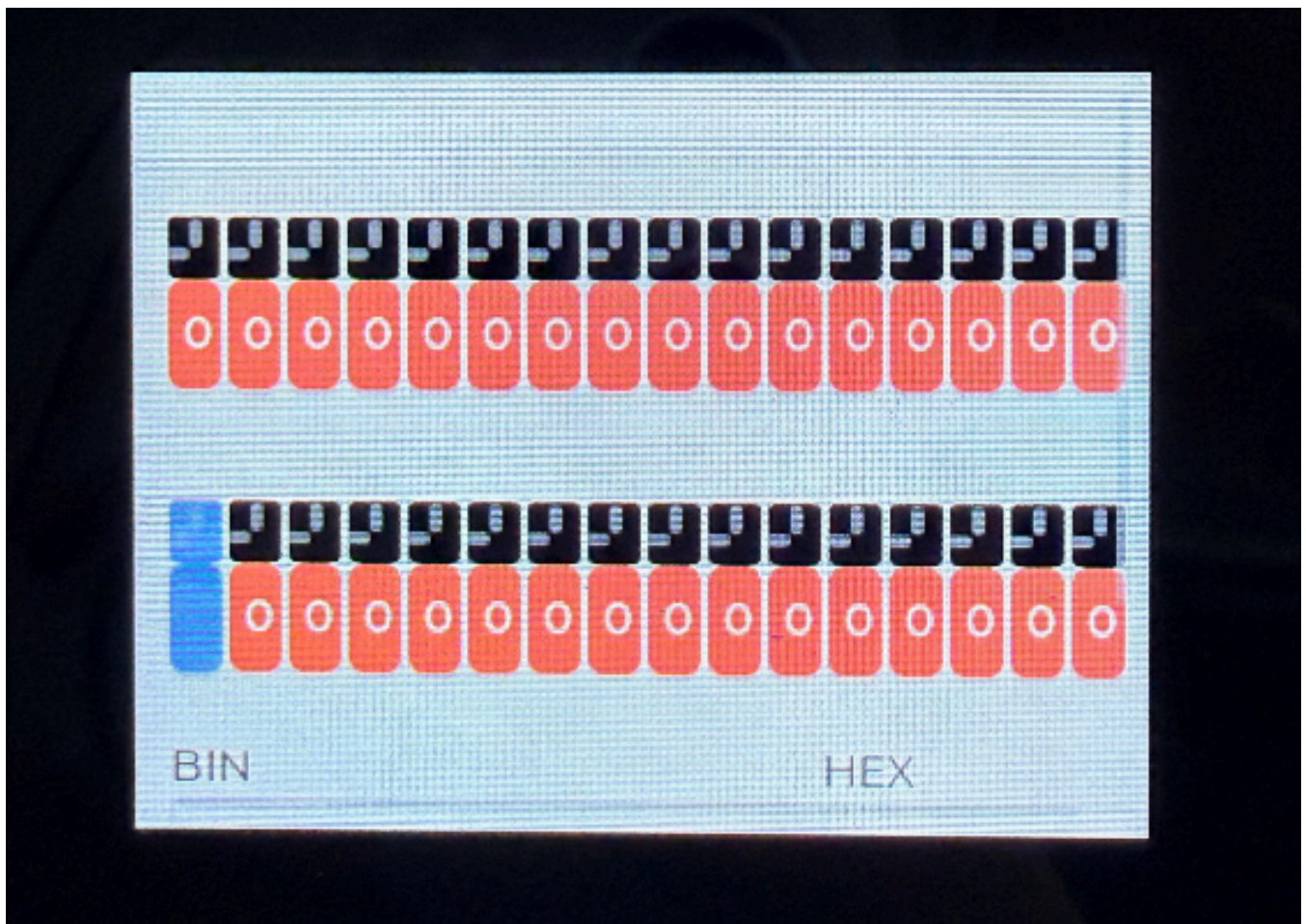
Note

Ensure the program file path contains only English characters, as non-English characters may cause compilation failures.

For specific Program Download Instructions (Must Read), refer to [5.1 Program Download Instructions \(Must Read\)](#).

5.14.3 Program Outcome

Upon program execution, the upper section of the screen presents a 33-bit binary button group, while the lower section features a layout containing four text input fields labeled BIN, DEC, HEX, and OCT. Clicking a binary bit button toggles its displayed state between "0" and "1". The system automatically converts the 33-bit binary value into decimal, hexadecimal, and octal formats, displaying the results in real-time within the placeholders of the corresponding input fields. Clicking any input field triggers a virtual keyboard adapted to that specific numeral system. For instance, the BIN input field enables only the "0" and "1" keys, while the DEC input field disables letter keys. Once a value is entered into a field, the system immediately performs mutual conversion across all bases and updates the placeholder content of the other input fields. The virtual keyboard can be dismissed by clicking the close or confirm icon. Throughout the operation, conversion results remain synchronized, and input rules strictly adhere to the character requirements of each numeral system. Additionally, clicking the blue button resets the input data to zero. The specific effect is illustrated in the figure below:



5.14.4 Program Analysis

Note

`lv_mainstart.c` is the file in the LVGL application project that most intuitively reflects the experiment's effects. Therefore, this chapter focuses on analyzing the program within `lv_mainstart.c` to help directly understand the core functions of the entire project.

- `lv_mainstart.c`

1. `lv_mainstart.c`: Based on LVGL, this file implements a multi-base interactive numerical interface. By utilizing 31 binary bit buttons alongside BIN, DEC, HEX, and OCT input fields, the system achieves real-time conversion and synchronized display of values across different numeral systems. The core logic of the program revolves around binary bit operations, text input handling, base conversion algorithms, and UI event coordination.

```
void lv_mainstart(void)
{
    memset(lv_bit_flag, '0', 31);
    lv_bit_flag[31] = '\0';

    lv_scale_cont = lv_obj_create(lv_scr_act());
    lv_obj_set_size(lv_scale_cont, lcddev.width, lcddev.height);
    lv_obj_set_pos(lv_scale_cont, 0, 0);

    lv_btnmatrix = NULL;
    lv_atk_scale_bit_init(lv_scale_cont);
}
```

2. `lv_atk_bit_str_reverse`: This section defines a series of static utility functions used for conversions between binary, decimal, hexadecimal, and octal formats. Specifically, the binary string reversal function serves to correct bit order issues, forming the foundation for subsequent calculations.

```
static void lv_atk_bit_str_reverse(char str[])
{
    int n = (int)strlen(str);
    for (int i = 0; i < n / 2; i++)
    {
        char t = str[i];
        str[i] = str[n - i - 1];
        str[n - i - 1] = t;
    }
}
```

3. `lv_atk_bit_bin_to_dec`: This function converts binary strings into decimal values. It parses the data bit-by-bit using a left-shift and accumulation method, featuring clear logic while avoiding the use of unsafe library functions.

```
static long lv_atk_bit_bin_to_dec(const char *pbin)
{
    long r = 0;
    for (int i = 0; pbin[i] != 0; i++)
        r = (r << 1) + (pbin[i] - '0');
    return r;
}
```

4. `lv_atk_dec_to_bin_str`: This function converts decimal numbers into binary strings of up to 31 bits. It employs a "low-order generation + reverse copy" approach, effectively preventing overflow issues.

```

static void lv_atk_dec_to_bin_str(long n, char *bin_str, int max_len)
{
    if (n == 0)
    {
        bin_str[0] = '0';
        bin_str[1] = '\0';
        return;
    }

    char temp[32] = {0};
    int idx = 0;

    while (n > 0 && idx < max_len - 1)
    {
        temp[idx++] = (n & 1) ? '1' : '0';
        n >>= 1;
    }

    for (int i = 0; i < idx; i++)
        bin_str[i] = temp[idx - 1 - i];
    bin_str[idx] = '\0';
}

```

5. `lv_update_bit_buttons_from_bin_str`: This function implements the core synchronization mechanism for "Text Input → Binary Bit Button Display." It updates the display status of the 31 buttons bit-by-bit based on the input binary string.

```

static void lv_update_bit_buttons_from_bin_str(const char *bin_str)
{
    if (!bin_str || bin_str[0] == '\0')
    {
        for (int i = 0; i < 31; i++)
        {
            lv_label_set_text(lv_bit_label[i], "0");
            lv_bit_flag[i] = '0';
        }
        return;
    }

    int len = strlen(bin_str);
    if (len > 31) len = 31;

    for (int i = 0; i < 31; i++)
    {
        if (i < len && bin_str[len - 1 - i] == '1')
        {
            lv_label_set_text(lv_bit_label[i], "1");
            lv_bit_flag[i] = '1';
        }
        else
        {

```

```

        lv_label_set_text(lv_bit_label[i], "0");
        lv_bit_flag[i] = '0';
    }
}
}

```

6. `lv_event_bit_map_handler`: Serving as one of the core event callbacks for the interface, this function handles binary bit button click events. Its responsibilities include bit toggling, processing the blue clear button, and triggering the coordinated update across all four numeral systems.

```

static void lv_event_bit_map_handler(lv_event_t *event)
{
    lv_event_code_t code = lv_event_get_code(event);
    lv_obj_t *obj = lv_event_get_target(event);

    if(code == LV_EVENT_CLICKED)
    {
        static char lv_buf[32];
        int lv_bit_post = 0;
        char lv_str_buf[32];
        char lv_oct_buf[32];
        char lv_bit_flag_buf[LV_ATK_BIT_COUNT] = "00000000000000000000000000000000";

        if (lv_btnmatrix)
        {
            lv_obj_del(lv_btnmatrix);
            if (lv_middle) lv_textarea_set_cursor_click_pos(lv_middle, true);
            lv_middle = NULL;
            lv_btnmatrix = NULL;
        }

        // Set flag to prevent VALUE_CHANGED event loop on input fields
        lv_updating_from_button = true;

        // Clear input field text only, not placeholders (avoids triggering VALUE_CHANGED)
        lv_textarea_set_text(lv_bin, "");
        lv_textarea_set_text(lv_dec, "");
        lv_textarea_set_text(lv_hex, "");
        lv_textarea_set_text(lv_oct, "");

        lv_bit_post = lv_atk_bit_to_dec(obj);

        // Check if it is the blue clear button (Index 31)
        if (lv_bit_post == 31)
        {
            // Clear function: Set all binary buttons (Index 0-30) to 0
            for (int i = 0; i < 31; i++) // Clear only 31 binary buttons
            {
                lv_label_set_text(lv_bit_label[i], "0");
                lv_bit_flag[i] = '0';
            }
        }
    }
}

```

```

        // Clear placeholders for all input fields
        lv_textarea_set_placeholder_text(lv_bin, "");
        lv_textarea_set_placeholder_text(lv_dec, "");
        lv_textarea_set_placeholder_text(lv_hex, "");
        lv_textarea_set_placeholder_text(lv_oct, "");
        // Reset flag
        lv_updating_from_button = false;
        return; // Return directly after clearing, skip subsequent update logic
    }

    lv_snprintf(lv_buf, sizeof(lv_buf), "%s",
lv_label_get_text(lv_bit_label[lv_bit_post]));

    if (strcmp(lv_buf, "1") == 0)
    {
        lv_label_set_text(lv_bit_label[lv_bit_post], "0");
        lv_bit_flag[lv_bit_post] = '0';
    }
    else if (strcmp(lv_buf, "0") == 0)
    {
        lv_label_set_text(lv_bit_label[lv_bit_post], "1");
        lv_bit_flag[lv_bit_post] = '1';
    }

    strncpy(lv_bit_flag_buf, lv_bit_flag, LV_ATK_BIT_COUNT - 1);
    lv_bit_flag_buf[LV_ATK_BIT_COUNT - 1] = '\0';

    lv_atk_bit_str_reverse(lv_bit_flag_buf);

    lv_textarea_set_placeholder_text(lv_bin, lv_bit_flag_buf);
    const char * sbin = lv_textarea_get_placeholder_text(lv_bin);
    lv_snprintf(lv_str_buf, sizeof(lv_str_buf), "%ld", lv_atk_bit_bin_to_dec(sbin));
    lv_textarea_set_placeholder_text(lv_dec, lv_str_buf);

    long vdec = strtol(lv_str_buf, NULL, 10);
    lv_snprintf(lv_oct_buf, sizeof(lv_oct_buf), "%ld", lv_atk_dec_to_oct(vdec));
    lv_textarea_set_placeholder_text(lv_oct, lv_oct_buf);

    lv_snprintf(lv_str_buf, sizeof(lv_str_buf), "0x%lx", lv_atk_bit_bin_to_dec(sbin));
    lv_textarea_set_placeholder_text(lv_hex, lv_str_buf);

    // Reset flag
    lv_updating_from_button = false;
}
}

```

7. `lv_ta_cb_event_handler()` : This is the common callback function for the BIN, DEC, HEX, and OCT input fields. It manages keyboard pop-ups, monitors input changes, and facilitates real-time conversion and synchronization between the different numeral systems.

```

static void lv_ta_cb_event_handler(lv_event_t *event)

```

```

{
    lv_event_code_t code = lv_event_get_code(event);
    lv_obj_t *ta = lv_event_get_target(event);

    if(code == LV_EVENT_VALUE_CHANGED)
    {
        if (lv_updating_from_button)
            return;

        const char * s = lv_textarea_get_text(ta);
        char bin_buf[32];

        if (ta == lv_dec && s && s[0])
        {
            long v = strtol(s, NULL, 10);
            lv_atk_dec_to_bin_str(v, bin_buf, sizeof(bin_buf));
            lv_update_bit_buttons_from_bin_str(bin_buf);
            lv_textarea_set_placeholder_text(lv_bin, bin_buf);
        }
    }
}

```

8. `lv_atk_scale_bit_init()`: Responsible for creating the 31 binary bit buttons, the blue clear button, and the BIN, DEC, HEX, and OCT input fields. This function handles layout and event binding, serving as the core function for UI construction.

```

void lv_atk_scale_bit_init(lv_obj_t *parent)
{
    int cell_w = (lcddev.width - 6) / 16;
    if(cell_w < 12) cell_w = 12;
    int idx_h = LV_ATK_BIT_INDEX_H;
    int btn_h = LV_ATK_BIT_H;
    int x0 = 3;
    int y_idx0 = 30;
    int y_idx1 = 120;

    for (int i = 0; i < LV_ATK_BIT_COUNT; i++)
    {
        int row, col, x, y_index, y_btn;

        if (i < 31) // First 31 are binary buttons
        {
            if (i < 16) // Row 1: Index 0-15, total 16
            {
                row = 0;
                col = 15 - i; // Right to left: 15, 14, 13, ..., 0
            }
            else // Row 2: Index 16-30, total 15
            {
                row = 1;
                col = 15 - (i - 16); // Right to left: 15, 14, 13, ..., 1
            }
        }
    }
}

```

```

    }
    x = x0 + col * cell_w;
    y_index = (row == 0) ? y_idx0 : y_idx1;
    y_btn = y_index + idx_h;
}
else // Index 31 is the blue clear button, placed at Row 2, far left (col 0)
{
    row = 1;
    col = 0; // Row 2 far left
    x = x0 + col * cell_w;
    y_index = y_idx1;
    y_btn = y_index + idx_h;
}

lv_bit_index[i] = lv_obj_create(parent);
lv_obj_set_style_bg_color(lv_bit_index[i], lv_color_hex3(0x00), LV_PART_MAIN);
lv_obj_set_style_border_side(lv_bit_index[i], 0, LV_PART_MAIN);
lv_obj_set_style_text_color(lv_bit_index[i], lv_color_hex3(0xffffffff), LV_PART_MAIN);
lv_obj_set_style_radius(lv_bit_index[i], 2, LV_PART_MAIN);
lv_obj_set_size(lv_bit_index[i], cell_w - 2, idx_h);
lv_obj_set_pos(lv_bit_index[i], x, y_index);

lv_bit_text[i] = lv_btn_create(parent);
lv_obj_set_style_radius(lv_bit_text[i], 5, LV_PART_MAIN);
lv_obj_set_style_bg_color(lv_bit_text[i], lv_palette_main(LV_PALETTE_RED),
LV_PART_MAIN);
lv_obj_set_style_border_side(lv_bit_text[i], 0, LV_PART_MAIN);
lv_obj_set_style_text_color(lv_bit_text[i], lv_color_hex3(0xffffffff), LV_PART_MAIN);
lv_obj_set_size(lv_bit_text[i], cell_w - 2, btn_h);
lv_obj_set_pos(lv_bit_text[i], x, y_btn);
lv_obj_add_event_cb(lv_bit_text[i], lv_event_bit_map_handler, LV_EVENT_ALL, NULL);

lv_bit_label[i] = lv_label_create(lv_bit_text[i]);
lv_label_set_text(lv_bit_label[i], "0");
lv_obj_center(lv_bit_label[i]);

if (i == 31) // Index 31 is the blue clear button
{
    lv_obj_set_style_bg_color(lv_bit_text[31], lv_palette_main(LV_PALETTE_BLUE),
LV_PART_MAIN);
    lv_obj_set_style_bg_color(lv_bit_index[31], lv_palette_main(LV_PALETTE_BLUE),
LV_PART_MAIN);
    lv_label_set_text(lv_bit_label[31], ""); // Clear button has no text
}
}

// BIN Input Field Position and Size (Left aligned with binary buttons, x0 = 3)
int bin_x = x0; // Left aligned with binary buttons
int bin_y = y_idx1 + btn_h + 20 + 20 + 20;
int bin_w = (int)(lcddev.width * 0.75) + 30; // width: 75% of screen width + 30px

// BIN Label placed outside top-left of input field (Above input field)
lv_bin_label = lv_label_create(parent);

```

```

lv_label_set_text(lv_bin_label, "BIN");
lv_obj_set_pos(lv_bin_label, bin_x, bin_y - 18); // Outside top-left, above input field

lv_bin = lv_textarea_create(parent);
lv_textarea_set_accepted_chars(lv_bin, "01");
lv_obj_set_style_text_font(lv_bin, &lv_font_montserrat_14, LV_PART_MAIN);
lv_obj_set_style_radius(lv_bin, 0, LV_PART_MAIN);
lv_obj_set_size(lv_bin, bin_w, LV_ATK_BIT_H);
lv_textarea_set_text(lv_bin, "");
lv_textarea_set_one_line(lv_bin, true);
lv_textarea_set_cursor_click_pos(lv_bin, true);
lv_obj_add_event_cb(lv_bin, lv_ta_cb_event_handler, LV_EVENT_ALL, NULL);
lv_obj_set_pos(lv_bin, bin_x, bin_y);

// DEC Input Field Position and Size (Left aligned with binary buttons, x0 = 3)
int dec_x = x0; // Left aligned with binary buttons
int dec_y = y_idx1 + btn_h + 20 + 20 + LV_ATK_BIT_H + 20 + 20;
int dec_w = (int)(lcddev.width * 0.75) + 30; // Width: 75% of screen width + 30px

// DEC Label placed outside top-left of input field (Above input field)
lv_dec_label = lv_label_create(parent);
lv_label_set_text(lv_dec_label, "DEC");
lv_obj_set_pos(lv_dec_label, dec_x, dec_y - 18); // Outside top-left, above input field

lv_dec = lv_textarea_create(parent);
lv_textarea_set_accepted_chars(lv_dec, "0123456789");
lv_obj_set_style_text_font(lv_dec, &lv_font_montserrat_14, LV_PART_MAIN);
lv_obj_set_style_radius(lv_dec, 0, LV_PART_MAIN);
lv_obj_set_size(lv_dec, dec_w, LV_ATK_BIT_H);
lv_textarea_set_text(lv_dec, "");
lv_textarea_set_one_line(lv_dec, true);
lv_textarea_set_cursor_click_pos(lv_dec, true);
lv_obj_add_event_cb(lv_dec, lv_ta_cb_event_handler, LV_EVENT_ALL, NULL);
lv_obj_set_pos(lv_dec, dec_x, dec_y);

// HEX Input Field Position and Size
int hex_x = lcddev.width/2 + 50;
int hex_y = y_idx1 + btn_h + 20 + 20 + 20;
int hex_w = lcddev.width / 2 - 60;

// HEX Label placed outside top-left of input field (Above input field)
lv_hex_label = lv_label_create(parent);
lv_label_set_text(lv_hex_label, "HEX");
lv_obj_set_pos(lv_hex_label, hex_x, hex_y - 18); // Outside top-left, above input field

lv_hex = lv_textarea_create(parent);
lv_textarea_set_accepted_chars(lv_hex, "0123456789ABCDEFabcdef");
lv_obj_set_style_text_font(lv_hex, &lv_font_montserrat_14, LV_PART_MAIN);
lv_obj_set_style_radius(lv_hex, 0, LV_PART_MAIN);
lv_obj_set_size(lv_hex, hex_w, LV_ATK_BIT_H);
lv_textarea_set_text(lv_hex, "");
lv_textarea_set_one_line(lv_hex, true);
lv_textarea_set_cursor_click_pos(lv_hex, true);

```

```

lv_obj_add_event_cb(lv_hex, lv_ta_cb_event_handler, LV_EVENT_ALL, NULL);
lv_obj_set_pos(lv_hex, hex_x, hex_y);

// OCT Input Field Position and Size
int oct_x = lcddev.width/2 + 50;
int oct_y = y_idx1 + btn_h + 20 + 20 + LV_ATK_BIT_H + 20 + 20;
int oct_w = lcddev.width / 2 - 60;

// OCT Label placed outside top-left of input field (Above input field)
lv_oct_label = lv_label_create(parent);
lv_label_set_text(lv_oct_label, "OCT");
lv_obj_set_pos(lv_oct_label, oct_x, oct_y - 18); // Outside top-left, above input field

lv_oct = lv_textarea_create(parent);
lv_textarea_set_accepted_chars(lv_oct, "01234567");
lv_obj_set_style_text_font(lv_oct, &lv_font_montserrat_14, LV_PART_MAIN);
lv_obj_set_style_radius(lv_oct, 0, LV_PART_MAIN);
lv_obj_set_size(lv_oct, oct_w, LV_ATK_BIT_H);
lv_textarea_set_text(lv_oct, "");
lv_textarea_set_one_line(lv_oct, true);
lv_textarea_set_cursor_click_pos(lv_oct, true);
lv_obj_add_event_cb(lv_oct, lv_ta_cb_event_handler, LV_EVENT_ALL, NULL);
lv_obj_set_pos(lv_oct, oct_x, oct_y);
}

```

